TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Bài tập lớn:

BrewLite – Assignment 2

REQUIREMENTS TO DEPLOYMENT

VER 1.0

Môn học:	Công nghệ Phần mềm
Công nghệ:	Next.js, NestJS, PostgreSQL và Docker
Học kỳ:	I – Năm học 2026–2027

GVHD: ThS. Từ Lãng Phiêu

Sinh viên thực hiện:

Kiệt	(Nhóm trưởng)	– 3124410175
Luân	(Thành viên)	– 3124410204
Hùng	(Thành viên)	– 3124410107
Triệu	(Thành viên)	– 3124410376
Lượng	(Thành viên)	– 3124410207
Kha	(Thành viên)	– 3124410136

TP. HỒ CHÍ MINH, NĂM 2026

Mục lục

1 Tổng quan Assignment 2 **3**
1.1 Phạm vi sản phẩm . . . 3

2 Quy trình từ yêu cầu đến triển khai **3**

3 Task 1 – Khai thác và đặc tả yêu cầu **3**
3.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi . . . 4
3.1.1 Bối cảnh và vấn đề nghiệp vụ . . . 4
3.1.2 Stakeholder . . . 4
3.1.3 Phạm vi . . . 5
3.1.4 Giả định . . . 6
3.1.5 Ràng buộc . . . 6
3.1.6 Rủi ro ban đầu . . . 6
3.2 Task 1.2 – Yêu cầu chức năng và phi chức năng . . . 7
3.2.1 Yêu cầu chức năng . . . 7
3.2.2 Yêu cầu phi chức năng . . . 12
3.2.3 Sơ đồ use-case tổng quát . . . 14
3.3 Task 1.3 – Đặc tả một use-case trọng tâm . . . 15
3.3.1 Bảng đặc tả use-case: Đặt và thanh toán đơn hàng . . . 15
3.3.2 Các luồng thực hiện (Flow of events) . . . 15
3.3.3 Sơ đồ use-case . . . 17
3.4 Task 1.4 – Rà soát chất lượng yêu cầu và lưu trữ dữ liệu . . . 17
3.4.1 Yêu cầu về dữ liệu và lưu trữ . . . 17
3.4.2 Checklist kiểm định chất lượng đặc tả yêu cầu . . . 18

4 Task 2 – Mô hình hóa hệ thống **18**
4.1 Task 2.1 – Activity diagram . . . 19
4.2 Task 2.2 – Sequence diagram . . . 21
4.3 Task 2.3 – Class diagram . . . 22
4.3.1 Sơ đồ lớp tổng quát . . . 23
4.3.2 Chi tiết sản phẩm và giỏ hàng . . . 24
4.3.3 Chi tiết đơn hàng và thanh toán . . . 25

5 Task 3 – Thiết kế kiến trúc **25**
5.1 Task 3.1 – Phương pháp và quyết định kiến trúc . . . 26
5.1.1 Phương pháp kiến trúc . . . 26
5.1.2 Các quyết định kiến trúc (ADR) . . . 26
5.2 Task 3.2 – Deployment diagram . . . 27

6 Task 4 – Triển khai Sprint 1 **28**
6.1 Task 4.1 – Thiết lập repository . . . 28
6.2 Task 4.2 – Quản lý tài liệu . . . 29
6.3 Task 4.3 – MVP Sprint 1 . . . 29
6.3.1 Frontend Next.js (apps/web) . . . 29
6.3.2 Backend NestJS & Cơ sở dữ liệu (apps/api) . . . 29
6.3.3 Kiểm thử tự động (Testing) . . . 30
6.4 Task 4.4 – Continuous Integration . . . 30
6.4.1 Chi tiết các Jobs trong Pipeline . . . 30
6.4.2 Chính sách Bảo vệ Nhánh (Branch Protection Rules) & Minh chứng Thực thi CI . . . 30

7 Task 5 – Triển khai Sprint 2, CI/CD và Delivery **32**
7.1 Task 5.1 – Hoàn thiện MVP . . . 32
7.2 Task 5.2 – Tích hợp và kiểm thử . . . 33
7.3 Task 5.3 – Continuous Delivery/Deployment . . . 34
7.3.1 Sơ đồ Chuỗi Pipeline Giao vận Đầy đủ . . . 34
7.3.2 Chi tiết Bảy Tiêu chí Vận hành Bắt buộc . . . 34
7.4 Task 5.4 – Triển khai và bàn giao . . . 35
7.4.1 Hạ tầng Môi trường Production & URL Hoạt động . . . 35
7.4.2 Phiên bản Phát hành và Release Notes . . . 36
7.4.3 Sổ tay Vận hành và Quy trình Khôi phục Sự cố (Rollback Playbook) . . . 36
7.4.4 Kịch bản Video Demo Trực tiếp Luồng End-to-End . . . 36
7.4.5 Đánh giá và Hồi tưởng Sprint (Sprint Review & Sprint Retrospective) . . . 36

8 Truy vết **37**
8.1 Truy vết bắt buộc . . . 37

9 Cấu trúc repository **38**
9.1 Checklist nộp bài . . . 39

1 Tổng quan Assignment 2

Assignment 2 tiếp tục phát triển đề tài **BrewLite** – ứng dụng đặt nước không tiền mặt – đã được khảo sát ở Assignment 1. Nhóm thực hiện toàn bộ chuỗi công việc từ khai thác yêu cầu, mô hình hóa, thiết kế kiến trúc đến hiện thực qua hai Sprint, thiết lập CI/CD và triển khai hệ thống lên môi trường production có thể truy cập được.

Báo cáo này trình bày kết quả của từng task, bắt đầu từ Task 1 (đặc tả yêu cầu) đến Task 5 (triển khai và bàn giao), kèm theo ma trận truy vết giữa yêu cầu, thiết kế, mã nguồn và kiểm thử.

1.1 Phạm vi sản phẩm

Luồng nghiệp vụ tối thiểu mà BrewLite phải đáp ứng:

Xem menu → **Chọn tùy chọn** → **Giỏ hàng** → **Đặt đơn** → **Thanh toán** → **Theo dõi đơn**

Hệ thống được xây dựng trên ba thành phần công nghệ bắt buộc:

- **Frontend:** Next.js theo hướng mobile-first/PWA.
- **Backend:** NestJS (TypeScript, REST API).
- **Cơ sở dữ liệu:** PostgreSQL.

Ngoài ra, hệ thống sử dụng Docker để đóng gói và đồng bộ môi trường, GitHub Actions cho CI/CD, và dịch vụ thanh toán mô phỏng để kiểm thử luồng thanh toán.

2 Quy trình từ yêu cầu đến triển khai

Dự án được chia thành năm task thực hiện tuần tự. Task sau chỉ được nghiệm thu khi các tạo phẩm bắt buộc của task trước đã có trên kho mã nguồn với lịch sử thay đổi rõ ràng.

Task	Giai đoạn	Tạo phẩm chính	Điểm
1	Khai thác và đặc tả yêu cầu	Stakeholder, phạm vi, SRS, use-case tổng quát và đặc tả use-case	2
2	Mô hình hóa hệ thống	Activity diagram, sequence diagram và class diagram	2
3	Thiết kế kiến trúc	Mô tả kiến trúc, quyết định kiến trúc và deployment diagram	2
4	Triển khai Sprint 1	Repository, tài liệu, CI cơ bản và MVP đầu tiên	2
5	Sprint 2 và Delivery	MVP hoàn chỉnh, CD, production, kiểm thử và bàn giao	2
		Tổng cộng	**10**

3 Task 1 – Khai thác và đặc tả yêu cầu

Task 1 tập trung vào việc xác định bối cảnh nghiệp vụ, nhận diện các bên liên quan, xây dựng tài liệu đặc tả yêu cầu phần mềm (SRS) rút gọn và đặc tả chi tiết một use-case trọng tâm. Đây là nền tảng để các task tiếp theo – mô hình hóa, thiết kế, hiện thực – có cơ sở thực hiện đúng hướng.

3.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi

3.1.1 Bối cảnh và vấn đề nghiệp vụ

Nhiều quán nước và cà phê quy mô nhỏ tại Việt Nam vẫn tiếp nhận đơn hàng chủ yếu qua hình thức gọi trực tiếp tại quầy. Quy trình này tồn tại một số vấn đề:

- Khách hàng phải xếp hàng chờ đợi, đặc biệt trong khung giờ cao điểm (giờ nghỉ trưa, chiều tối).
- Nhân viên ghi đơn thủ công dễ nhầm lẫn sản phẩm, tùy chọn (size, topping) hoặc số lượng.
- Thanh toán tiền mặt gây bất tiện: thiếu tiền thối, không kiểm soát doanh thu tức thì.
- Không có hệ thống lưu trữ lịch sử đơn hàng, khó phân tích dữ liệu kinh doanh và hành vi khách hàng.
- Khách hàng không biết trạng thái đơn hàng (đang pha chế, sẵn sàng nhận) sau khi đặt.

BrewLite ra đời nhằm giải quyết các vấn đề trên bằng cách cung cấp một ứng dụng web đặt nước trực tuyến, hỗ trợ thanh toán không tiền mặt. Khách hàng có thể tự xem menu, chọn sản phẩm cùng tùy chọn, đặt đơn và thanh toán qua thiết bị di động mà không cần tương tác trực tiếp tại quầy. Nhân viên pha chế nhận đơn chính xác từ hệ thống và cập nhật trạng thái để khách hàng theo dõi.

3.1.2 Stakeholder

Bảng 1 liệt kê các bên liên quan chính của hệ thống BrewLite cùng mục tiêu và kỳ vọng.

Bảng 1: Bảng stakeholder hệ thống BrewLite

Stakeholder	**Vai trò**	**Mục tiêu và kỳ vọng**
Khách hàng	Người dùng cuối, đặt và thanh toán đơn hàng	Đặt nước nhanh chóng, chọn size/topping dễ dàng, thanh toán không tiền mặt, theo dõi trạng thái đơn hàng và xem lại lịch sử đơn.
Nhân viên quầy	Tiếp nhận và pha chế đơn hàng	Nhận đơn hàng chính xác từ hệ thống, cập nhật trạng thái pha chế (đang chuẩn bị, sẵn sàng giao), giảm sai sót do ghi chép thủ công.
Quản lý cửa hàng	Quản lý vận hành và menu sản phẩm	Quản lý danh mục sản phẩm, theo dõi đơn hàng tổng thể, đảm bảo hệ thống hoạt động ổn định phục vụ kinh doanh.
Giảng viên / Khách hàng nghiệm thu	Đánh giá sản phẩm theo tiêu chí môn học	Hệ thống đáp ứng đầy đủ yêu cầu đề bài, có tài liệu rõ ràng, mã nguồn có kiểm thử, pipeline CI/CD hoạt động và URL production truy cập được.
Nhóm phát triển	Thiết kế, hiện thực và triển khai hệ thống	Áp dụng quy trình công nghệ phần mềm, chia task hợp lý, sử dụng Git/GitHub hiệu quả, hoàn thành đúng hạn và học hỏi công nghệ mới.
Đơn vị thanh toán	Cung cấp dịch vụ thanh toán (SePay sandbox)	API thanh toán hoạt động ổn định, nhận webhook (IPN) chính xác, thông tin giao dịch được ghi nhận đầy đủ.

3.1.3 Phạm vi

Phạm vi MVP (In-scope)

Các tính năng nằm trong phạm vi MVP mà nhóm cam kết hoàn thành:

1. **Xem menu sản phẩm:** Hiển thị danh sách sản phẩm với tên, hình ảnh, giá bán.
2. **Xem chi tiết và chọn tùy chọn:** Xem thông tin chi tiết sản phẩm, chọn size và topping.
3. **Quản lý giỏ hàng:** Thêm, cập nhật số lượng, xóa sản phẩm trong giỏ hàng.
4. **Xác thực người dùng:** Đăng ký, đăng nhập, đăng xuất với JWT.
5. **Đặt đơn hàng:** Tạo đơn hàng từ giỏ hàng hợp lệ, kiểm tra tồn kho.
6. **Thanh toán mô phỏng:** Lựa chọn phương thức (Ví/Thẻ), xử lý và xác nhận kết quả.
7. **Theo dõi và quản lý đơn:** Xem trạng thái, lịch sử đơn hàng; nhân viên cập nhật trạng thái pha chế.
8. **Triển khai:** Docker, CI/CD qua GitHub Actions, URL production truy cập được qua HTTPS.

Phạm vi mở rộng (Future scope)

Các tính năng có thể phát triển thêm sau MVP nhưng không bắt buộc trong phạm vi dự án:

- Hệ thống tích điểm khách hàng thân thiết (loyalty program).
- Mã giảm giá và chương trình khuyến mãi.
- Giao diện quản trị cho quản lý (admin panel): thêm/sửa/ẩn sản phẩm.
- Báo cáo và thống kê doanh thu.
- Thông báo đẩy (push notification) khi đơn hàng thay đổi trạng thái.
- Đánh giá và nhận xét sản phẩm từ khách hàng.

Ngoài phạm vi (Out-of-scope)

Các nội dung nhóm **không** thực hiện trong dự án này:

- Giao hàng tận nơi (delivery) – hệ thống chỉ hỗ trợ nhận tại quầy.
- Thanh toán bằng tiền thật – toàn bộ thanh toán là mô phỏng (sandbox).
- Hỗ trợ nhiều chi nhánh cửa hàng (multi-store).
- Ứng dụng native trên iOS/Android – chỉ có PWA trên trình duyệt.
- Tích hợp với hệ thống kế toán hoặc ERP bên ngoài.
- Hỗ trợ đa ngôn ngữ (i18n).

3.1.4 Giả định

1. Cửa hàng hoạt động với một chi nhánh duy nhất.
2. Khách hàng sử dụng thiết bị di động hoặc máy tính có kết nối Internet và trình duyệt hiện đại (Chrome, Safari, Firefox).
3. Thanh toán hoàn toàn là mô phỏng thông qua SePay sandbox; không xử lý tiền thật.
4. Menu sản phẩm được nhập sẵn qua database seed; trong MVP, quản lý không chỉnh sửa menu qua giao diện.
5. PostgreSQL và các dịch vụ hạ tầng (Docker, GitHub Actions) luôn khả dụng trong quá trình phát triển và triển khai.
6. Nhóm phát triển gồm 6 thành viên làm việc bán thời gian (sinh viên).
7. Khách hàng cần đăng nhập trước khi đặt đơn và thanh toán.

3.1.5 Ràng buộc

1. **Công nghệ bắt buộc:** Frontend phải dùng Next.js, backend phải dùng NestJS, dữ liệu lưu trong PostgreSQL. Không được thay thế ba thành phần này.
2. **Thời gian:** Dự án phải hoàn thành theo lịch trình môn Công nghệ Phần mềm, học kỳ I năm học 2026–2027.
3. **Nhân lực:** Nhóm gồm 6 sinh viên, mỗi thành viên dành thời gian có giới hạn ngoài giờ học.
4. **Thanh toán:** Chỉ mô phỏng, tuyệt đối không sử dụng thông tin thanh toán thật hoặc xử lý giao dịch tài chính thực tế.
5. **Quy trình:** Năm task phải được thực hiện tuần tự; task sau chỉ nghiệm thu khi task trước hoàn thành.
6. **Bảo mật:** Không đưa secret (JWT secret, mật khẩu database, khóa thanh toán) lên GitHub hoặc vào log CI/CD.
7. **Triển khai:** Hệ thống phải có URL production truy cập được qua HTTPS tại thời điểm nghiệm thu.

3.1.6 Rủi ro ban đầu

Bảng 2 phân tích các rủi ro chính có thể ảnh hưởng đến dự án cùng biện pháp giảm thiểu.

Bảng 2: Bảng phân tích rủi ro ban đầu

Mã	Rủi ro	Xác suất	Tác động	Biện pháp giảm thiểu
R-01	Thành viên chưa quen với Next.js hoặc NestJS	Cao	Trung bình	Chia sẻ tài liệu học nội bộ, pair programming, code review hỗ trợ lẫn nhau.
R-02	Xung đột mã nguồn (merge conflict) khi nhiều người cùng sửa	Trung bình	Trung bình	Phân nhánh theo tính năng, mỗi thành viên làm trên branch riêng, review trước khi merge.

Mã	Rủi ro	Xác suất	Tác động	Biện pháp giảm thiểu
R-03	Deadline gấp, khối lượng công việc lớn so với nhân lực	Cao	Cao	Ưu tiên tính năng MVP, cắt giảm tính năng phụ, theo dõi tiến độ hàng tuần qua bảng backlog.
R-04	API sandbox thanh toán (SePay) thay đổi hoặc ngừng hoạt động	Thấp	Cao	Xây dựng mock/stub thanh toán dự phòng để kiểm thử không phụ thuộc dịch vụ bên ngoài.
R-05	Mất dữ liệu database khi triển khai hoặc migration	Thấp	Cao	Sao lưu database trước mỗi lần migration, sử dụng migration có thể rollback.
R-06	Thành viên không hoàn thành task đúng hạn	Trung bình	Cao	Họp tiến độ ngắn hàng tuần, phát hiện sớm điểm nghẽn, sẵn sàng phân công lại kịp thời.

3.2 Task 1.2 – Yêu cầu chức năng và phi chức năng

3.2.1 Yêu cầu chức năng

FR-01 – Xem menu sản phẩm

Mô tả: Hệ thống cho phép khách hàng khám phá thực đơn và xem danh sách các món đồ uống đang được cung cấp theo từng danh mục.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Mỗi sản phẩm trên danh sách hiển thị đầy đủ: hình ảnh minh họa, tên sản phẩm, mô tả ngắn gọn và mức giá cơ sở (giá khởi điểm cho size tiêu chuẩn).
- Hỗ trợ bộ lọc danh mục (Tabs: Cà phê, Trà sữa, Trà trái cây, Đá xay...); phản hồi tức thì trên cùng trang (SPA) không tải lại trang.
- Hiển thị hiệu ứng tải (Loading / Skeleton) trong khi chờ dữ liệu nạp từ Backend API `GET /products`.
- Các sản phẩm hết hàng hoặc ngừng kinh doanh không được hiển thị như sản phẩm có thể đặt hàng.

FR-02 – Xem chi tiết sản phẩm

Mô tả: Khách hàng có thể mở cửa sổ chi tiết (Modal) của một sản phẩm để tùy biến kích cỡ (size) theo khẩu vị.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Khi chọn một sản phẩm, hệ thống mở Modal hiển thị tên, hình ảnh, mô tả chi tiết, giá gốc và các kích cỡ hỗ trợ (Size S, M, L).
- Bắt buộc chọn duy nhất 1 kích cỡ tại một thời điểm (Single choice). Mặc định hệ thống tự chọn Size tiêu chuẩn (Size M).
- Đơn giá hiển thị tự động cộng hoặc trừ chênh lệch phụ thu tương ứng theo từng kích cỡ.

FR-03 – Chọn tùy chọn sản phẩm và tính giá động

Mô tả: Khách hàng có thể chọn nhiều loại topping thêm cho ly nước và theo dõi tổng tiền tạm tính tự động theo thời gian thực.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Cho phép chọn cùng lúc nhiều loại topping khác nhau (Multiple choice: Trân châu, Thạch đào, Kem cheese...); chặn hoàn toàn số lượng âm.
- Tổng tiền tạm tính ở chân Modal được tính toán tự động theo thời gian thực:

$$\text{Tổng tiền} = \left(\text{Giá cơ sở} + \text{Phụ thu Size} + \sum \text{Giá Topping}\right) \times \text{Số lượng ly}$$

- Khi bấm nút "Thêm vào giỏ hàng", toàn bộ cấu hình món đã chọn được đóng gói chính xác vào giỏ hàng và tự động đóng Modal.

FR-04 – Thêm sản phẩm vào giỏ hàng

Mô tả: Khách hàng có thể thêm sản phẩm cùng các tùy chọn đã chọn vào giỏ hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Sản phẩm được thêm vào giỏ hàng với đúng số lượng, size, topping và đơn giá tương ứng.

FR-05 – Cập nhật giỏ hàng

Mô tả: Khách hàng có thể thay đổi số lượng hoặc tùy chọn sản phẩm trong giỏ hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống cập nhật số lượng, thành tiền của từng sản phẩm và tổng tiền giỏ hàng chính xác sau mỗi thay đổi. Số lượng phải là số nguyên dương.

FR-06 – Xóa sản phẩm khỏi giỏ hàng

Mô tả: Khách hàng có thể xóa sản phẩm không muốn mua khỏi giỏ hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Sản phẩm được loại khỏi giỏ hàng và tổng tiền được tính lại chính xác.

FR-07 – Kiểm tra giỏ hàng trước khi đặt đơn

Mô tả: Hệ thống kiểm tra thông tin sản phẩm, số lượng, tùy chọn và giá trước khi tạo đơn hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống chỉ cho phép tiếp tục khi giỏ hàng có ít nhất một sản phẩm hợp lệ. Nếu sản phẩm không còn khả dụng hoặc giá đã thay đổi, hệ thống thông báo cho khách hàng và yêu cầu xác nhận thông tin mới khi cần thiết.

FR-08 – Nhập thông tin nhận đơn

Mô tả: Khách hàng cung cấp thông tin cần thiết để cửa hàng tiếp nhận và xử lý đơn hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống kiểm tra các trường thông tin bắt buộc theo hình thức nhận đơn được hỗ trợ. Nếu dữ liệu không hợp lệ hoặc còn thiếu, hệ thống thông báo lỗi cụ thể.

FR-09 – Xem lại thông tin đơn hàng

Mô tả: Trước khi đặt đơn, khách hàng có thể kiểm tra các sản phẩm, tùy chọn, số lượng và tổng tiền.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống hiển thị đầy đủ thông tin đơn hàng và tổng tiền được tính từ dữ liệu hợp lệ ở phía máy chủ.

FR-10 – Tạo đơn hàng

Mô tả: Hệ thống tạo đơn hàng từ giỏ hàng hợp lệ của khách hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Đơn hàng được cấp mã định danh duy nhất.
- Đơn hàng lưu thông tin sản phẩm, số lượng, tùy chọn, đơn giá và tổng tiền tại thời điểm đặt.
- Hệ thống kiểm tra lại dữ liệu trước khi lưu đơn hàng.
- Đơn hàng không được tạo trùng do cùng một yêu cầu gửi lại ngoài ý muốn.
- Nếu thao tác lưu đơn thất bại, hệ thống thông báo kết quả phù hợp và không để lại dữ liệu đơn hàng dở dang.

FR-11 – Theo dõi trạng thái đơn hàng

Mô tả: Khách hàng có thể xem trạng thái hiện tại của đơn hàng đã đặt.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống hiển thị mã đơn hàng, thời điểm đặt và trạng thái hiện tại. Thông tin hiển thị phải khớp với dữ liệu được lưu trên hệ thống.

FR-12 – Cập nhật trạng thái đơn hàng

Mô tả: Nhân viên hoặc người có quyền phù hợp có thể cập nhật trạng thái đơn hàng theo quy trình xử lý của cửa hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Chỉ người có quyền mới được cập nhật trạng thái. Hệ thống từ chối các chuyển đổi trạng thái không hợp lệ và lưu trạng thái mới khi cập nhật thành công.

Các trạng thái dự kiến:

- Chờ thanh toán.
- Đã xác nhận.
- Đang chuẩn bị.
- Sẵn sàng giao/nhận.
- Hoàn thành.
- Đã hủy.

Việc chuyển trạng thái phải tuân theo quy tắc nghiệp vụ. Trạng thái thanh toán được quản lý riêng, không mặc định đồng nhất với trạng thái xử lý đơn hàng.

FR-13 – Xem lịch sử đơn hàng

Mô tả: Khách hàng có thể xem lại các đơn hàng đã đặt thuộc tài khoản của mình.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hệ thống hiển thị danh sách đơn hàng của đúng khách hàng, bao gồm mã đơn, thời điểm đặt, tổng tiền và trạng thái. Khách hàng không được truy cập lịch sử đơn hàng của tài khoản khác.

FR-14 – Đăng ký tài khoản

Mô tả: Hệ thống cho phép khách hàng tạo tài khoản bằng cách cung cấp họ tên, email và mật khẩu.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Người dùng đăng ký thành công khi cung cấp đầy đủ thông tin hợp lệ.
- Email đăng ký không được trùng với email đã tồn tại.
- Mật khẩu phải có tối thiểu 8 ký tự.
- Hệ thống hiển thị thông báo khi đăng ký thành công hoặc thất bại.
- Mật khẩu được mã hóa bằng thuật toán băm an toàn trước khi lưu vào cơ sở dữ liệu.

FR-15 – Đăng nhập tài khoản

Mô tả: Hệ thống cho phép người dùng đăng nhập bằng email và mật khẩu đã đăng ký.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Người dùng đăng nhập thành công khi email và mật khẩu chính xác.
- Hệ thống trả về JWT hợp lệ sau khi xác thực thành công.
- Khi thông tin đăng nhập không chính xác, hệ thống hiển thị thông báo lỗi phù hợp.
- Người dùng chưa đăng nhập không thể thực hiện các thao tác yêu cầu xác thực.

FR-16 – Đăng xuất tài khoản

Mô tả: Hệ thống cho phép người dùng đăng xuất khỏi phiên làm việc hiện tại.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Khi chọn đăng xuất, hệ thống xóa thông tin xác thực của phiên hiện tại ở phía ứng dụng.
- Người dùng được chuyển về trạng thái chưa đăng nhập.
- Người dùng phải đăng nhập lại để tiếp tục sử dụng các chức năng được bảo vệ.
- JWT đã được cấp trước đó có thể vẫn hợp lệ đến khi hết hạn nếu hệ thống chưa triển khai cơ chế thu hồi token.

FR-17 – Xác thực truy cập bằng JWT

Mô tả: Backend NestJS kiểm tra JWT trước khi cho phép người dùng truy cập các API yêu cầu xác thực.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- API được bảo vệ chỉ xử lý yêu cầu khi JWT hợp lệ.
- JWT không hợp lệ, hết hạn hoặc không được cung cấp sẽ bị từ chối với mã HTTP 401.
- Người dùng không được truy cập thông tin đơn hàng thuộc tài khoản khác.
- Hệ thống kiểm tra quyền truy cập trước khi thực hiện thao tác trên tài nguyên.

FR-18 – Lựa chọn phương thức thanh toán

Mô tả: Khách hàng có thể lựa chọn một trong hai phương thức thanh toán mô phỏng: Ví điện tử hoặc Thẻ.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Hệ thống hiển thị đầy đủ hai phương thức thanh toán.
- Khách hàng phải chọn một phương thức trước khi gửi yêu cầu thanh toán.
- Phương thức được chọn phải được ghi nhận cùng thông tin thanh toán của đơn hàng.

FR-19 – Xử lý thanh toán mô phỏng

Mô tả: Hệ thống gửi yêu cầu thanh toán đến dịch vụ thanh toán mô phỏng và tiếp nhận kết quả xử lý.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Chỉ cho phép thanh toán đối với đơn hàng hợp lệ.
- Số tiền thanh toán phải bằng tổng tiền đơn hàng được backend xác nhận.
- Hệ thống hỗ trợ mô phỏng cả kết quả thanh toán thành công và thất bại.
- Khi thanh toán thành công, giao dịch được ghi nhận với trạng thái thành công.
- Khi thanh toán thất bại, hệ thống ghi nhận trạng thái thất bại và hiển thị thông báo cho khách hàng.

FR-20 – Xác nhận kết quả thanh toán

Mô tả: Sau khi xử lý thanh toán, hệ thống thông báo kết quả và cập nhật trạng thái thanh toán của đơn hàng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Khi thanh toán thành công, hệ thống hiển thị thông báo xác nhận.
- Hệ thống lưu mã giao dịch, phương thức thanh toán, số tiền, thời gian và trạng thái thanh toán.
- Khi thanh toán thất bại, đơn hàng không được đánh dấu là đã thanh toán.
- Trạng thái thanh toán được đồng bộ với thông tin đơn hàng.

FR-21 – Ngăn chặn thanh toán trùng lặp

Mô tả: Hệ thống kiểm soát các yêu cầu thanh toán được gửi lặp lại nhằm tránh ghi nhận nhiều giao dịch thành công cho cùng một lần thanh toán.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Khi gửi lại yêu cầu với cùng mã idempotency key, hệ thống không tạo thêm giao dịch thanh toán mới.

- Hệ thống trả về kết quả đã ghi nhận của yêu cầu tương ứng.
- Một yêu cầu thanh toán đã hoàn tất không được xử lý thành công thêm lần nữa chỉ vì người dùng nhấn nút nhiều lần.

3.2.2 Yêu cầu phi chức năng

NFR-01 – Bảo vệ mật khẩu người dùng

Mô tả: Mật khẩu người dùng phải được bảo vệ trong quá trình lưu trữ và xác thực.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Mật khẩu được băm bằng thuật toán phù hợp như bcrypt hoặc Argon2.
- Không lưu mật khẩu dưới dạng văn bản thuần.
- API không trả về mật khẩu hoặc giá trị băm mật khẩu.
- Kiểm thử cơ sở dữ liệu xác nhận không tồn tại mật khẩu dạng văn bản thuần.

NFR-02 – Bảo mật truyền dữ liệu

Mô tả: Dữ liệu trao đổi giữa trình duyệt, backend và các dịch vụ bên ngoài phải được bảo vệ.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Môi trường production sử dụng giao thức HTTPS.
- Thông tin đăng nhập và thanh toán không được truyền qua HTTP công khai.
- Các yêu cầu không an toàn phải được chuyển hướng sang HTTPS hoặc từ chối theo cấu hình triển khai.

NFR-03 – Kiểm soát truy cập

Mô tả: Hệ thống phải giới hạn quyền truy cập dựa trên trạng thái xác thực và quyền của người dùng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Người dùng chưa đăng nhập không thể truy cập API quản lý đơn hàng cá nhân.
- Khách hàng chỉ có thể xem những đơn hàng thuộc tài khoản của mình.
- Các thao tác dành cho nhân viên hoặc quản lý phải được kiểm tra quyền trên backend.
- Yêu cầu không đủ quyền bị từ chối với mã HTTP 403.

NFR-04 – Kiểm tra và xác thực dữ liệu đầu vào

Mô tả: Hệ thống phải kiểm tra dữ liệu được gửi từ frontend trước khi xử lý.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Backend kiểm tra kiểu dữ liệu, định dạng và các trường bắt buộc.
- Các dữ liệu không hợp lệ bị từ chối với mã HTTP 400.
- Dữ liệu do người dùng nhập không được sử dụng trực tiếp để xây dựng câu lệnh SQL không an toàn.
- Giá trị thanh toán không được tin cậy hoàn toàn từ frontend mà phải được backend xác minh.

NFR-05 – Bảo vệ thông tin cấu hình và dữ liệu nhạy cảm

Mô tả: Hệ thống phải bảo vệ các thông tin bí mật như JWT secret, thông tin kết nối PostgreSQL và khóa cấu hình dịch vụ.

Mức ưu tiên: Must.

Tiêu chí chấp nhận:

- Secret được quản lý thông qua biến môi trường hoặc dịch vụ quản lý bí mật.
- Không đưa mật khẩu database hoặc JWT secret thật lên GitHub.
- Các thông tin nhạy cảm không xuất hiện trong log hệ thống hoặc log CI/CD.
- Cơ sở dữ liệu production không được công khai trực tiếp trên Internet.

NFR-06 – Ghi nhận và theo dõi sự kiện bảo mật

Mô tả: Hệ thống ghi nhận những sự kiện cần thiết phục vụ kiểm tra lỗi và phát hiện hành vi truy cập bất thường.

Mức ưu tiên: Should.

Tiêu chí chấp nhận:

- Ghi nhận các lần đăng nhập thất bại và lỗi xử lý thanh toán.
- Log có thời gian, loại sự kiện và thông tin định danh phù hợp.
- Không ghi mật khẩu, JWT đầy đủ hoặc thông tin thẻ nhạy cảm vào log.
- Có thể tra cứu log để xác định nguyên nhân khi một giao dịch thanh toán gặp lỗi.

NFR-07 – Hiệu năng: Thời gian phản hồi API

Mô tả: Hệ thống Backend (NestJS) phải phản hồi các yêu cầu từ Client (Next.js) một cách nhanh chóng để tối ưu trải nghiệm người dùng.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: 95% các yêu cầu API GET (xem menu, danh sách đơn hàng) phản hồi dưới 2 giây. Các API ghi dữ liệu (POST/PUT như tạo đơn, thanh toán) xử lý dưới 3 giây trong điều kiện mạng bình thường.

NFR-08 – Hiệu năng: Khả năng chịu tải đồng thời

Mô tả: Hệ thống NestJS và CSDL PostgreSQL phải đảm bảo hoạt động ổn định trong các khung giờ cao điểm khi lượng đặt hàng tăng đột biến.

Mức ưu tiên: Should.

Tiêu chí chấp nhận: Hệ thống duy trì hoạt động với tỷ lệ lỗi 5xx dưới 1% khi chịu tải đồng thời tối thiểu 100 người dùng và xử lý thành công ít nhất 100 request/giây.

NFR-09 – Tính khả dụng: Giao diện Mobile-first & PWA

Mô tả: Giao diện Next.js được thiết kế tối ưu cho điện thoại di động và hỗ trợ các tính năng Progressive Web App (PWA).

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Giao diện hiển thị chuẩn xác, không bị vỡ bố cục trên màn hình di động có chiều rộng từ 375px trở lên và cho phép người dùng thêm ứng dụng vào màn hình chính (Add to Home Screen).

NFR-10 – Tính khả dụng: Phản hồi trạng thái giao diện

Mô tả: Giao diện người dùng phải phản hồi tức thì và cung cấp thông tin trực quan đối với mọi tương tác hoặc trạng thái chờ của hệ thống.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Hiển thị hiệu ứng chờ (Skeleton/Spinner) khi tải dữ liệu và hiển thị thông báo lỗi thân thiện kèm nút "Thử lại" (Retry) khi mất kết nối hoặc gọi API thất bại.

NFR-11 – Khả năng triển khai: Đóng gói Containerization

Mô tả: Toàn bộ các dịch vụ hệ thống (Next.js, NestJS, PostgreSQL) được đóng gói dưới dạng Docker container để đồng bộ môi trường.

Mức ưu tiên: Must.

Tiêu chí chấp nhận: Tất cả các dịch vụ được định nghĩa trong file docker-compose.yml và có thể khởi chạy hoàn chỉnh ở môi trường cục bộ chỉ với một lệnh docker compose up.

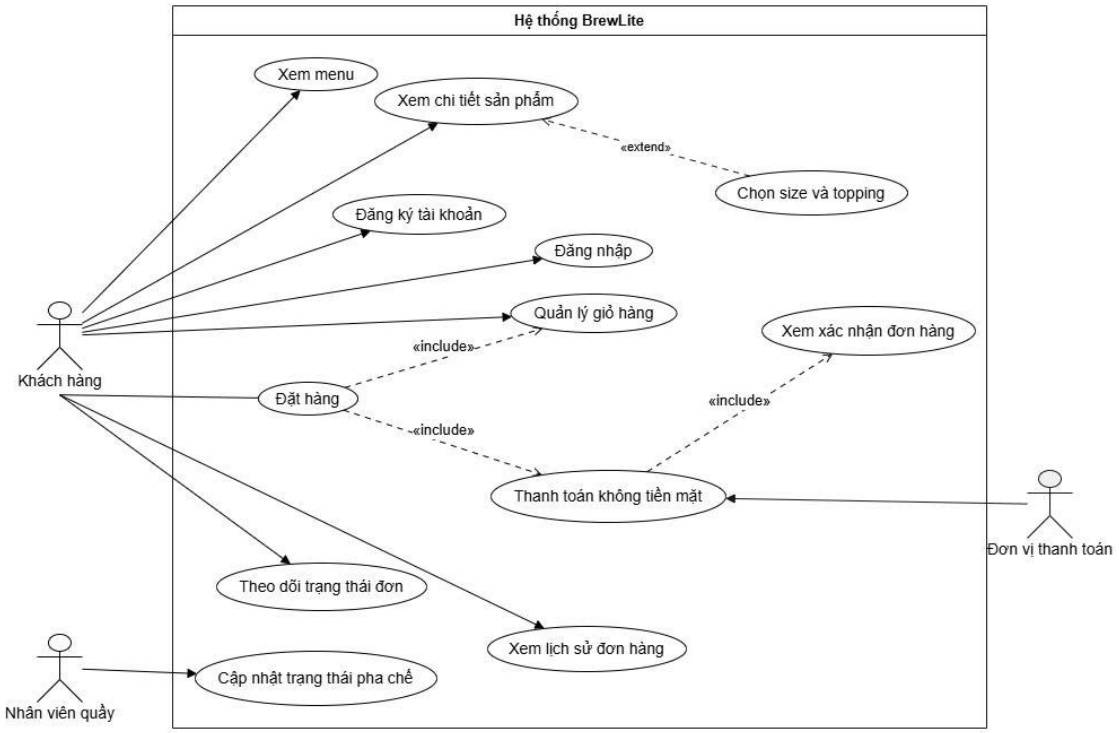
NFR-12 – Khả năng triển khai: Quy trình CI/CD & Health Check

Mô tả: Hệ thống tích hợp quy trình kiểm thử tự động qua GitHub Actions và cung cấp điểm cuối (endpoint) giám sát sức khỏe ứng dụng.

Mức ưu tiên: Should.

Tiêu chí chấp nhận: Pipeline CI tự động chặn merge Pull Request nếu quá trình Lint, Unit Test hoặc Build thất bại. Endpoint GET /health trả về mã trạng thái 200 OK khi các dịch vụ hoạt động bình thường.

3.2.3 Sơ đồ use-case tổng quát



Hình 1: Sơ đồ use-case tổng quát của hệ thống BrewLite

3.3 Task 1.3 – Đặc tả một use-case trọng tâm

3.3.1 Bảng đặc tả use-case: Đặt và thanh toán đơn hàng

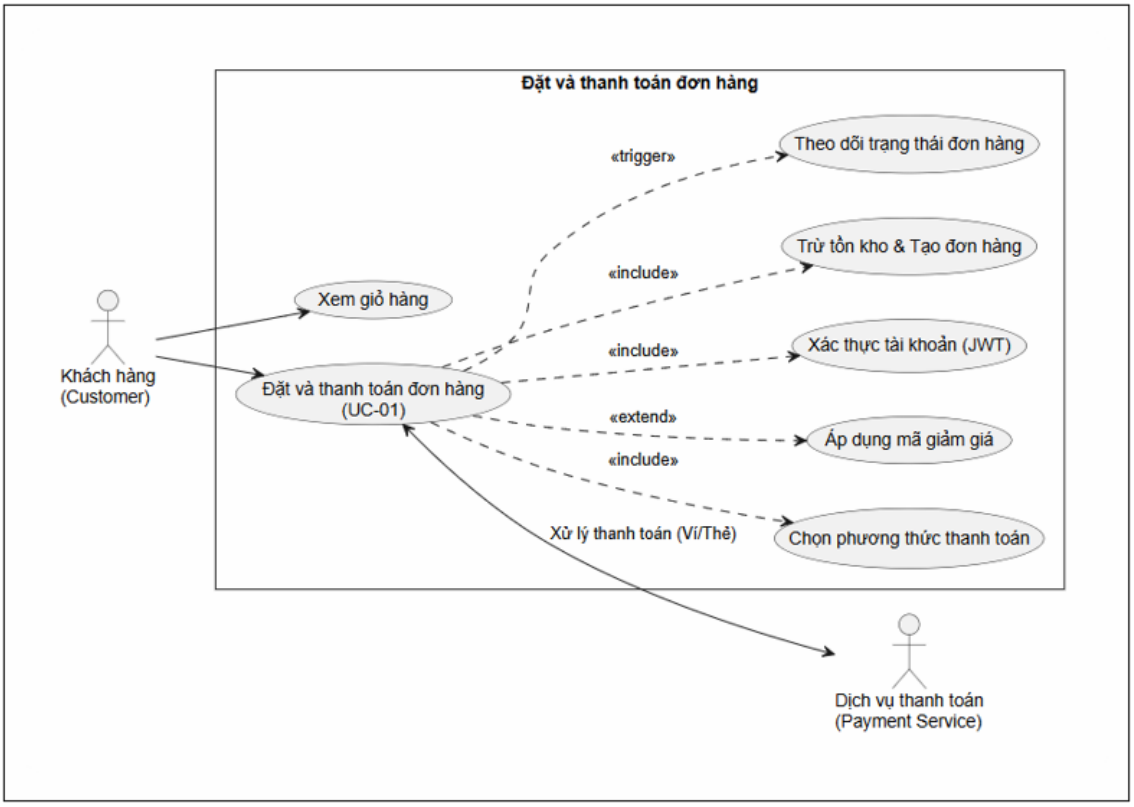
Thành phần	**Nội dung chi tiết**
Mã Use-case	UC-01
Tên Use-case	Đặt và thanh toán đơn
Actor chính	Khách hàng
Actor phụ / Hệ thống	• Hệ thống Backend NestJS API • Cơ sở dữ liệu PostgreSQL • Dịch vụ thanh toán (SePay Sandbox)
Trigger	Khách hàng nhấn nút "Thanh toán" từ màn hình giỏ hàng hoặc nhấn "Xác nhận đặt hàng".
Mô tả vắn tắt	Khách hàng kiểm tra giỏ hàng, xác nhận thông tin đơn hàng, chọn phương thức thanh toán và tiến hành thanh toán trực tuyến. Hệ thống thực hiện khởi tạo đơn, trừ số lượng tồn kho và cập nhật trạng thái đơn hàng.
Mã yêu cầu liên quan	• FR-04, FR-05, FR-07: Quản lý giỏ hàng • FR-08, FR-09, FR-10: Tạo và xác nhận đơn hàng • FR-17: Xác thực truy cập bằng JWT • FR-18, FR-19, FR-20, FR-21: Xử lý thanh toán và chống trùng lặp
Tiền điều kiện	1. Khách hàng đã đăng nhập vào ứng dụng (có token JWT hợp lệ). 2. Giỏ hàng có ít nhất 01 sản phẩm. 3. Dịch vụ thanh toán (Payment Gateway) đang hoạt động bình thường.
Hậu điều kiện	• Thành công: Bản ghi đơn hàng được tạo ở trạng thái PAID, số lượng tồn kho sản phẩm được trừ, giỏ hàng được làm sạch, khách hàng được chuyển đến màn hình theo dõi đơn. • Thất bại: Đơn hàng bị hủy (CANCELLED), tồn kho không thay đổi, giỏ hàng giữ nguyên và hiển thị thông báo lỗi phù hợp.

3.3.2 Các luồng thực hiện (Flow of events)

Loại luồng	**Bước thực hiện chi tiết**
Luồng chính	1. Khách hàng mở màn hình Giỏ hàng trên ứng dụng Next.js, kiểm tra danh sách món, tùy chọn (size, topping) và tổng tiền.
	2. Khách hàng nhấn nút "Thanh toán".
	3. Hệ thống hiển thị giao diện xác nhận thông tin đơn hàng và chọn phương thức thanh toán (Thẻ/Ví mô phỏng).
	4. Khách hàng chọn phương thức thanh toán và nhấn "Xác nhận đặt hàng".
	5. Frontend gửi yêu cầu POST /orders (kèm JWT Token, Idempotency-Key và dữ liệu đơn hàng) tới NestJS API.

Loại luồng	Bước thực hiện chi tiết
	6. NestJS API xác thực JWT, kiểm tra tính hợp lệ của dữ liệu và kiểm tra tồn kho trong PostgreSQL.
	7. NestJS API khởi tạo đơn hàng tạm thời (PENDING) và gọi Dịch vụ thanh toán (Payment Service).
	8. Payment Service xử lý thanh toán thành công và trả về mã giao dịch (Transaction ID).
	9. NestJS API cập nhật đơn hàng thành PAID, trừ tồn kho sản phẩm và lưu lịch sử thanh toán trong một transaction duy nhất.
	10. NestJS API trả về phản hồi thành công (HTTP 201 Created).
	11. Frontend xóa giỏ hàng địa phương và chuyển hướng Khách hàng sang màn hình Theo dõi đơn hàng.
Luồng thay thế	• AF-01 (Áp dụng mã giảm giá): Tại bước 3, khách hàng nhập mã ưu đãi hợp lệ. Hệ thống tính lại tổng tiền thanh toán trước khi khách hàng bấm xác nhận.
Luồng ngoại lệ / Lỗi	• EF-01 (Lỗi xác thực - 401 Unauthorized): Token JWT hết hạn hoặc không hợp lệ. Hệ thống báo lỗi và yêu cầu khách hàng đăng nhập lại.
	• EF-02 (Hết hàng - 400 Bad Request): Món/topping trong giỏ đã hết tồn kho khi kiểm tra tại bước 6. Hệ thống hủy thao tác và thông báo khách hàng cập nhật lại giỏ hàng.
	• EF-03 (Thanh toán thất bại): Cổng thanh toán trả về lỗi (không đủ tiền, lỗi kết nối). Hệ thống cập nhật đơn hàng thành CANCELLED, rollback tồn kho, giữ nguyên giỏ hàng và cho phép thử lại.
	• EF-04 (Yêu cầu trùng lặp): Khi nhận cùng một Idempotency-Key đã xử lý, backend trả về kết quả đơn hàng đã tạo trước đó mà không trừ tồn kho lần hai.

3.3.3 Sơ đồ use-case



Hình 2: Sơ đồ use-case đặt và thanh toán đơn hàng

3.4 Task 1.4 – Rà soát chất lượng yêu cầu và lưu trữ dữ liệu

3.4.1 Yêu cầu về dữ liệu và lưu trữ

Để đảm bảo tính toàn vẹn và nhất quán của hệ thống theo quy định đề bài, kiến trúc lưu trữ dữ liệu trên PostgreSQL phải tuân thủ các nguyên tắc sau:

1. **Snapshot dữ liệu khi tạo đơn hàng (Data Snapshotting):** Tại thời điểm tạo đơn hàng (FR-10), hệ thống phải snapshot toàn bộ tên món, giá bán, size và topping vào bảng `OrderItem`. Khi giá sản phẩm gốc trong bảng `Product` thay đổi trong tương lai, tổng tiền và chi tiết của các đơn hàng cũ tuyệt đối không bị ảnh hưởng.

2. **Toàn vẹn giao dịch (ACID Transaction):** Thao tác tạo đơn hàng, trừ số lượng tồn kho (`stock`) và lưu bản ghi thanh toán phải được thực thi trong một Database Transaction duy nhất. Nếu bất kỳ bước nào thất bại, toàn bộ quá trình phải được rollback tự động để tránh mất mát dữ liệu hoặc tạo đơn rác.

3. **Kiểm soát tương tranh tồn kho (Concurrency Control):** Khi nhiều người dùng cùng đặt mua sản phẩm có số lượng tồn kho giới hạn tại cùng thời điểm, hệ thống phải sử dụng cơ chế khóa phù hợp (Pessimistic Locking hoặc Optimistic Locking) nhằm đảm bảo số lượng tồn không bao giờ bị âm.

4. **Cơ chế Idempotency:** Lưu trữ khóa `idempotency_key` cùng với bản ghi giao dịch thanh toán nhằm ngăn chặn việc trừ tiền hoặc tạo đơn hai lần do lỗi mạng hoặc người dùng nhấn nút nhiều lần.

5. **Bảo mật dữ liệu định danh (Data Privacy):** Mật khẩu người dùng chỉ được lưu dưới dạng chuỗi băm (hash bằng bcrypt/Argon2). Khóa bí mật JWT Secret và SePay Secret Key không được lưu trong database mà phải lấy từ biến môi trường.

3.4.2 Checklist kiểm định chất lượng đặc tả yêu cầu

Bảng 5 trình bày kết quả rà soát chất lượng tài liệu đặc tả yêu cầu (SRS) theo các tiêu chí chuẩn kỹ nghệ phần mềm (IEEE 830 / ISO/IEC/IEEE 29148).

Bảng 5: Checklist rà soát chất lượng tài liệu yêu cầu

STT	Tiêu chí kiểm định	Nội dung kiểm tra thực tế trên SRS BrewLite	Kết quả
1	**Tính đầy đủ (Completeness)**	Toàn bộ 21 yêu cầu chức năng (FR-01 đến FR-21) và 12 yêu cầu phi chức năng (NFR-01 đến NFR-12) đều có mã định danh, mô tả chi tiết, độ ưu tiên MoSCoW và tiêu chí chấp nhận.	**Đạt**
2	**Tính nhất quán (Consistency)**	Không có sự trùng lặp mã yêu cầu; quy ước phân loại MoSCoW (Must, Should) được áp dụng đồng bộ xuyên suốt toàn tài liệu; thuật ngữ kỹ thuật thống nhất giữa frontend (Next.js), backend (NestJS) và database (PostgreSQL).	**Đạt**
3	**Tính rõ ràng (Unambiguity)**	Mỗi yêu cầu chỉ diễn đạt một ý định nghiệp vụ duy nhất, không dùng từ ngữ mơ hồ; tiêu chí chấp nhận nêu rõ điều kiện đầu vào và kết quả mong đợi (ví dụ: mã HTTP 400, 401, 201).	**Đạt**
4	**Khả năng kiểm chứng (Verifiability)**	Tất cả các tiêu chí chấp nhận có thể kiểm thử thông qua Unit Test, E2E Test hoặc kiểm tra thủ công (ví dụ: NFR-07 quy định 95% request GET dưới 2s, NFR-08 quy định chịu tải 100 req/s).	**Đạt**
5	**Khả năng truy vết (Traceability)**	Các yêu cầu chức năng cốt lõi (FR-04, FR-08, FR-10, FR-18) được ánh xạ trực tiếp sang use-case trọng tâm UC-01, sẵn sàng để liên kết vào ma trận truy vết ở Task 8.	**Đạt**
6	**Tính khả thi (Feasibility)**	Phạm vi yêu cầu được giới hạn ở mức MVP phù hợp với năng lực nhóm và thời hạn học kỳ; loại bỏ các tính năng phức tạp (delivery, multi-store, tiền thật).	**Đạt**

4 Task 2 – Mô hình hóa hệ thống

Task 2 xây dựng ba sơ đồ UML cốt lõi để mô hình hóa hành vi, tương tác và cấu trúc dữ liệu của hệ thống BrewLite: (1) Sơ đồ Hoạt động mô tả luồng nghiệp vụ end-to-end; (2) Sơ đồ Tuần tự

đặc tả chi tiết tương tác đa thành phần trong use-case trọng tâm UC-01; (3) Sơ đồ Lớp Lĩnh vực xác định mô hình thực thể và cấu trúc dữ liệu toàn hệ thống. Tất cả sơ đồ đều được lưu kèm tệp nguồn (PlantUML/draw.io) và ảnh kết xuất tại thư mục `docs/diagrams/`.

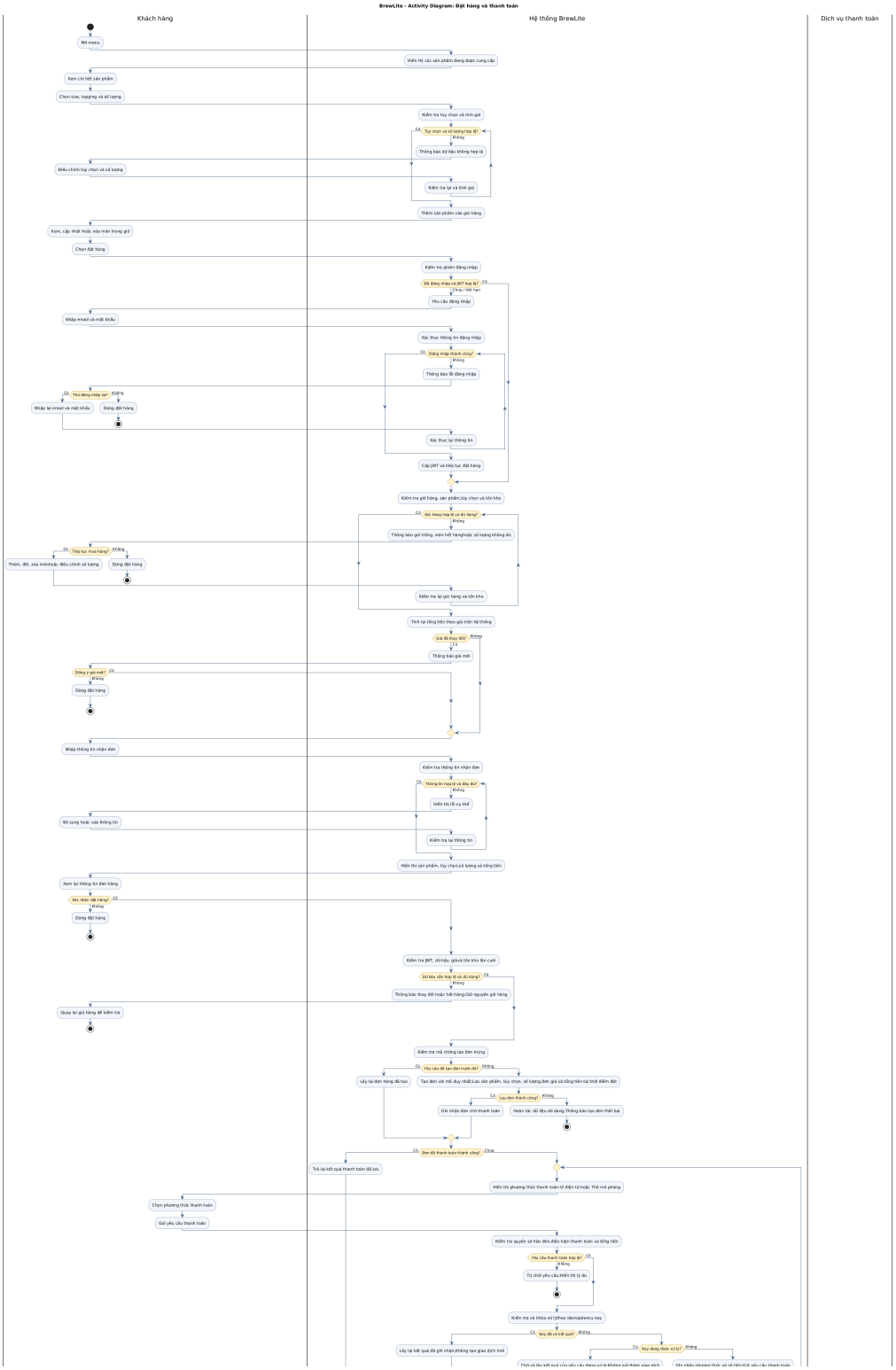
4.1 Task 2.1 – Activity diagram

Sơ đồ Hoạt động mô tả toàn bộ luồng nghiệp vụ từ khi khách hàng duyệt menu đến khi hoàn tất đơn hàng và theo dõi tiến độ pha chế, được tổ chức theo ba làn trách nhiệm (swimlanes):

- **Khách hàng (Customer):** Thực hiện duyệt thực đơn, chọn size và topping, quản lý giỏ hàng, nhập thông tin nhận đơn và xác nhận thanh toán.
- **Hệ thống BrewLite (Next.js & NestJS):** Kiểm tra phiên đăng nhập qua JWT, kiểm tra tồn kho (pessimistic lock `SELECT …FOR UPDATE`), tính lại tổng tiền, tạo đơn và kiểm soát Idempotency Key để ngăn ghi đơn trùng lặp.
- **Dịch vụ Thanh toán (SePay Sandbox):** Tiếp nhận yêu cầu thanh toán mô phỏng và phản hồi kết quả (Thành công / Thất bại) qua Webhook IPN.

Sơ đồ xử lý đầy đủ các nhánh rẽ nghiệp vụ và ngoại lệ:

- *Chưa đăng nhập / JWT hết hạn:* Yêu cầu đăng nhập trước khi tiếp tục; cho phép thử lại hoặc dừng luồng.
- *Hết hàng hoặc thay đổi giá:* Cảnh báo và yêu cầu khách xác nhận trước khi tiếp tục.
- *Chống tạo đơn trùng lặp:* Idempotency Key cho phép truy hồi đơn đã tạo thay vì tạo mới.
- *Thanh toán thất bại:* Khách hàng có thể thử lại với Idempotency Key mới hoặc chuyển sang phương thức khác.



Hình 3: Sơ đồ Hoạt động – Luồng đặt hàng và thanh toán BrewLite (UC-01)

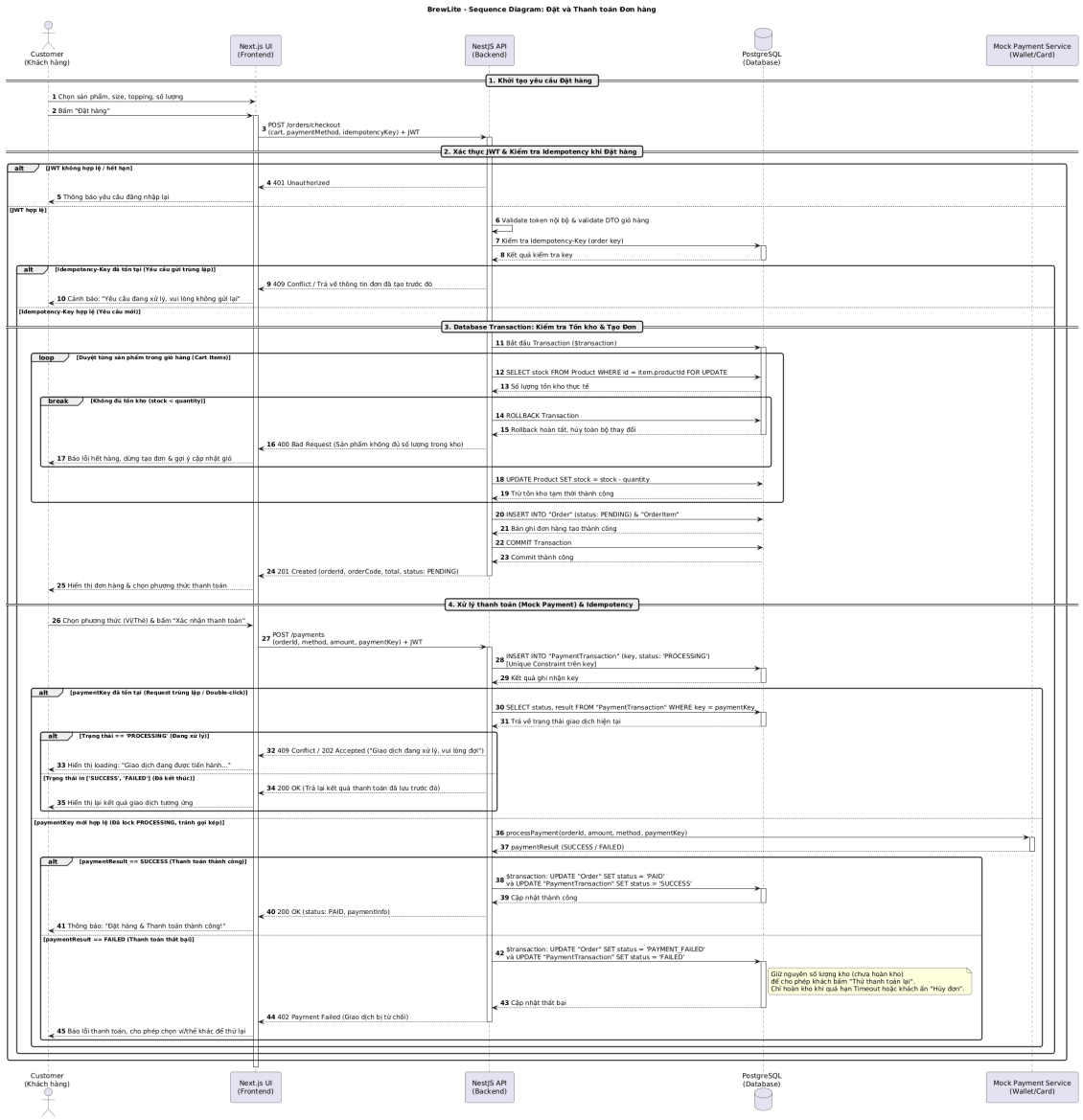
4.2 Task 2.2 – Sequence diagram

Sơ đồ Tuần tự đặc tả chi tiết sự tương tác theo trình tự thời gian giữa năm lifeline trong use-case trọng tâm **UC-01: Đặt và thanh toán đơn hàng**:

1. **Customer (Khách hàng)** – Tác nhân khởi phát yêu cầu.
2. **Next.js UI (Frontend)** – Giao diện người dùng, tổng hợp và gửi yêu cầu API.
3. **NestJS API (Backend)** – Lõi nghiệp vụ: xác thực, kiểm tra tồn kho, tạo đơn.
4. **PostgreSQL (Database)** – Nguồn dữ liệu chính xác nhận tồn kho và lưu đơn hàng.
5. **Mock Payment Service (SePay Sandbox)** – Cổng thanh toán mô phỏng.

Các điểm nhấn kỹ thuật quan trọng:

- **Xác thực JWT & Idempotency khi đặt hàng:** Mọi yêu cầu tạo đơn phải kèm JWT hợp lệ và `Idempotency-Key` duy nhất; hệ thống trả `409 Conflict` nếu key đã tồn tại thay vì tạo đơn trùng.
- **Database Transaction & Concurrency Control:** Toàn bộ luồng kiểm tra tồn kho và tạo đơn được bao trong `$transaction` Prisma. Mỗi sản phẩm được khóa bi quan (`SELECT …FOR UPDATE`) trước khi trừ stock; nếu không đủ hàng, thực hiện `ROLLBACK` ngay và trả `400 Bad Request`.
- **Xử lý Double-Click Lock khi thanh toán:** Bản ghi `PaymentTransaction` khởi tạo với trạng thái `PROCESSING` và Unique Constraint trên khóa giao dịch; gửi lặp bị trả về `409 Conflict` mà không gọi cổng thanh toán lần hai.
- **Nhánh thanh toán thành công:** Đơn hàng chuyển sang `PAID`, lưu mã giao dịch; **thất bại:** đơn sang `PAYMENT_FAILED`, kho hàng được giải phóng sau timeout cho phép thử lại.

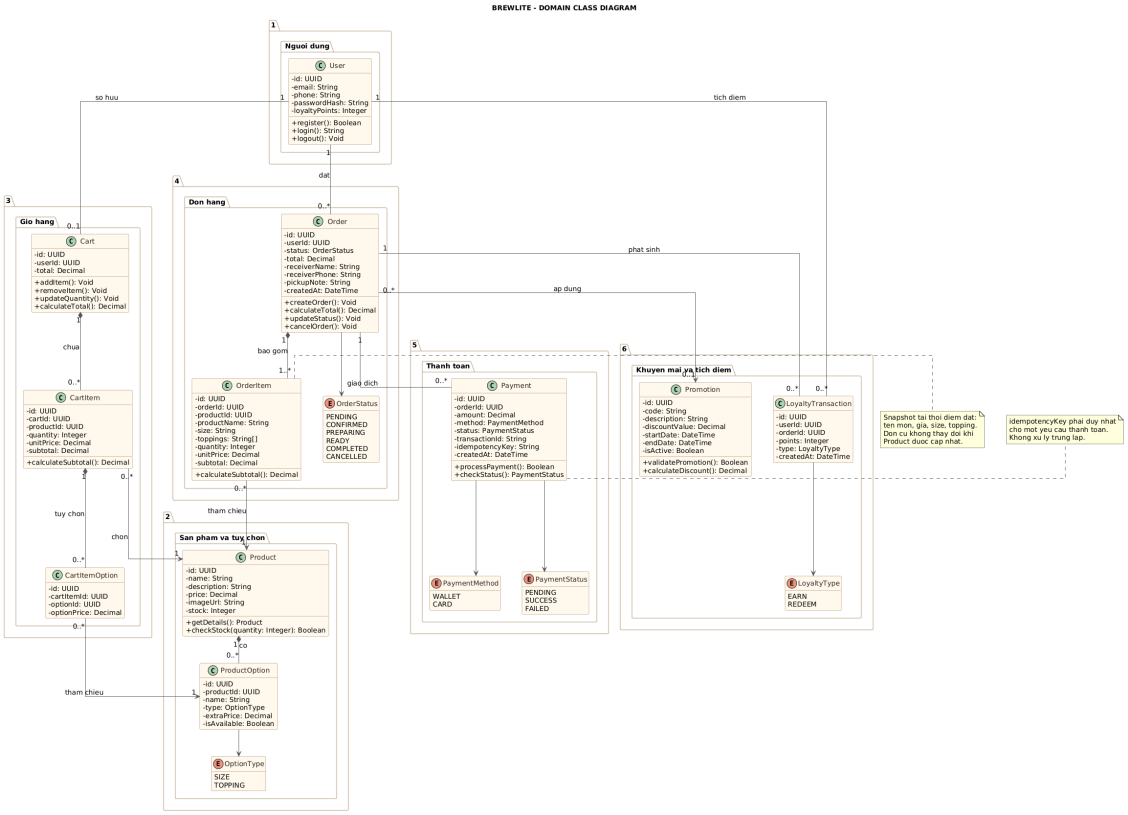


Hình 4: Sơ đồ Tuần tự – UC-01: Đặt và thanh toán đơn hàng (BrewLite)

4.3 Task 2.3 – Class diagram

Sơ đồ Lớp mô tả cấu trúc các đối tượng nghiệp vụ, thuộc tính, kiểu dữ liệu, phương thức và quan hệ trong hệ thống BrewLite theo mô hình miền (Domain Model). Toàn bộ mã nguồn PlantUML của sơ đồ lớp được lưu trữ tại `docs/diagrams/class-diagram.puml`.

4.3.1 Sơ đồ lớp tổng quát

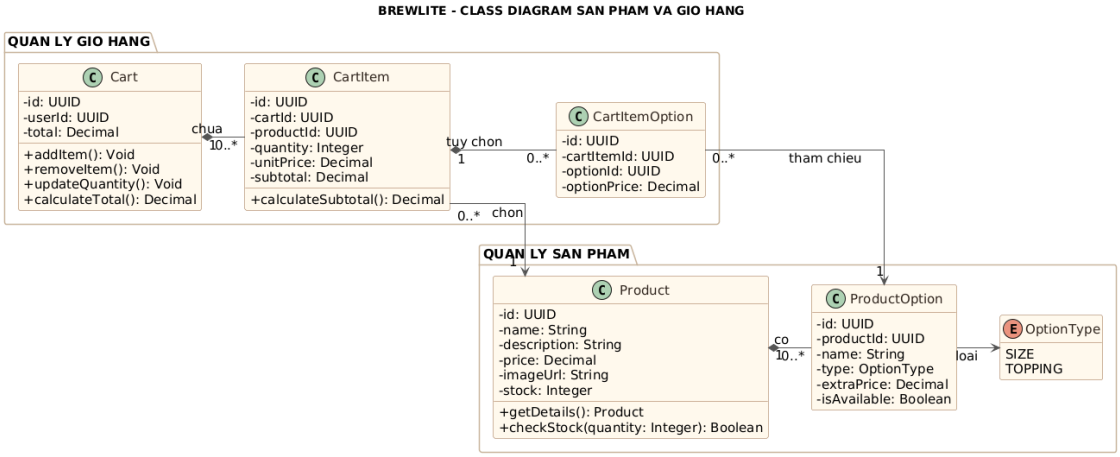


Hình 5: Sơ đồ Lớp tổng quát – Domain Model BrewLite (7 nhóm chức năng)

Hệ thống được tổ chức thành 6 nhóm lớp chức năng:

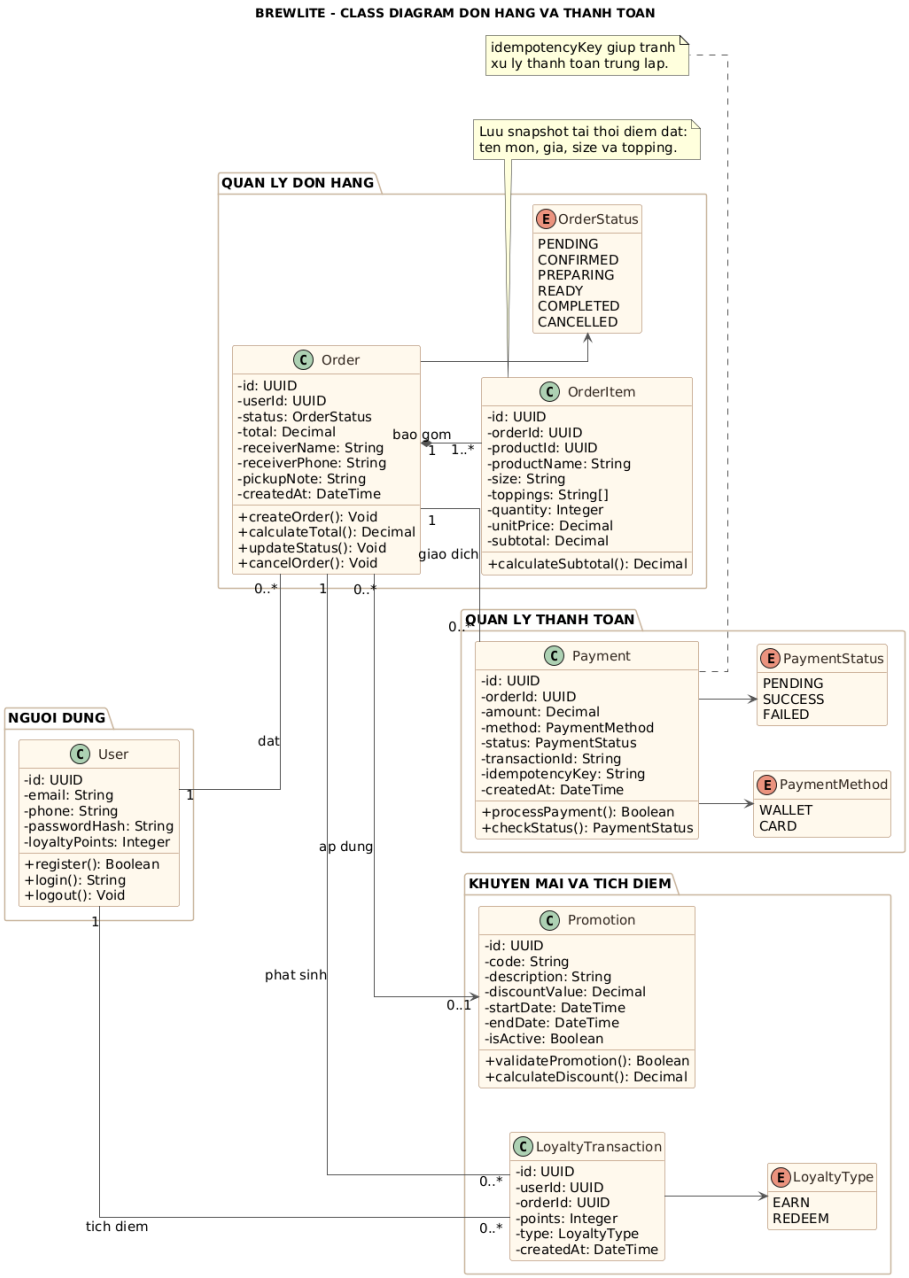
1. **Người dùng (User):** Lưu thông tin tài khoản, mật khẩu băm bằng bcrypt và điểm tích lũy.
2. **Sản phẩm & Tùy chọn (Product & ProductOption):** Quản lý đồ uống, giá bán, tồn kho, kích cỡ và topping.
3. **Giỏ hàng (Cart, CartItem, CartItemOption):** Quản lý giỏ hàng tạm thời và tùy chọn của từng mục.
4. **Đơn hàng (Order, OrderItem):** Lớp `OrderItem` áp dụng *Data Snapshotting* – lưu cố định tên, giá, size và topping tại thời điểm đặt; thay đổi giá sản phẩm gốc sau này không ảnh hưởng lịch sử đơn.
5. **Thanh toán (Payment):** Quản lý giao dịch mô phỏng, phương thức, trạng thái và khóa `idempotencyKey`.
6. **Khuyến mãi & Tích điểm (Promotion, LoyaltyTransaction):** Phục vụ mở rộng tích điểm và mã giảm giá.

4.3.2 Chi tiết sản phẩm và giỏ hàng



Hình 6: Sơ đồ Lớp chi tiết – Product, Cart và các lớp tùy chọn

4.3.3 Chi tiết đơn hàng và thanh toán



Hình 7: Sơ đồ Lớp chi tiết – Order, OrderItem và Payment

5 Task 3 – Thiết kế kiến trúc

Task 3 thiết kế kiến trúc tổng thể của hệ thống BrewLite, bao gồm: lựa chọn phương pháp kiến trúc, ghi nhận các quyết định kiến trúc quan trọng dưới dạng ADR (Architecture Decision Record) và vẽ sơ đồ UML Deployment Diagram thể hiện hạ tầng triển khai production thực tế.

5.1 Task 3.1 – Phương pháp và quyết định kiến trúc

5.1.1 Phương pháp kiến trúc

BrewLite áp dụng kiến trúc **3 tầng phân tách rõ ràng** (Layered / N-Tier Architecture), triển khai theo mô hình **monorepo** với hai ứng dụng độc lập trong cùng repository:

- **Tầng Presentation:** Next.js 14 PWA (`apps/web`) – giao diện người dùng mobile-first, React Server Components, App Router.
- **Tầng Nghiệp vụ:** NestJS 10 (`apps/api`) – logic miền, RBAC, Prisma ORM, middleware chống double-submit.
- **Tầng Dữ liệu:** PostgreSQL 16 – lưu trữ ACID, migration-first qua Prisma Migrate, khoá bi quan khi tạo đơn.
- **Dịch vụ bên ngoài:** SePay Sandbox Payment Gateway – thanh toán mô phỏng, IPN Webhook.

Các tầng giao tiếp qua giao thức HTTPS/REST (Frontend → Backend) và Prisma TCP/SSL nội bộ (Backend → Database). Kiến trúc tuân theo nguyên tắc *Separation of Concerns*: mỗi tầng chỉ phụ thuộc một chiều vào tầng kế tiếp.

5.1.2 Các quyết định kiến trúc (ADR)

Bốn quyết định kiến trúc quan trọng được ghi lại tại `docs/architecture/adr/`:

Bảng 6: Danh sách các Architecture Decision Records (ADR) – BrewLite

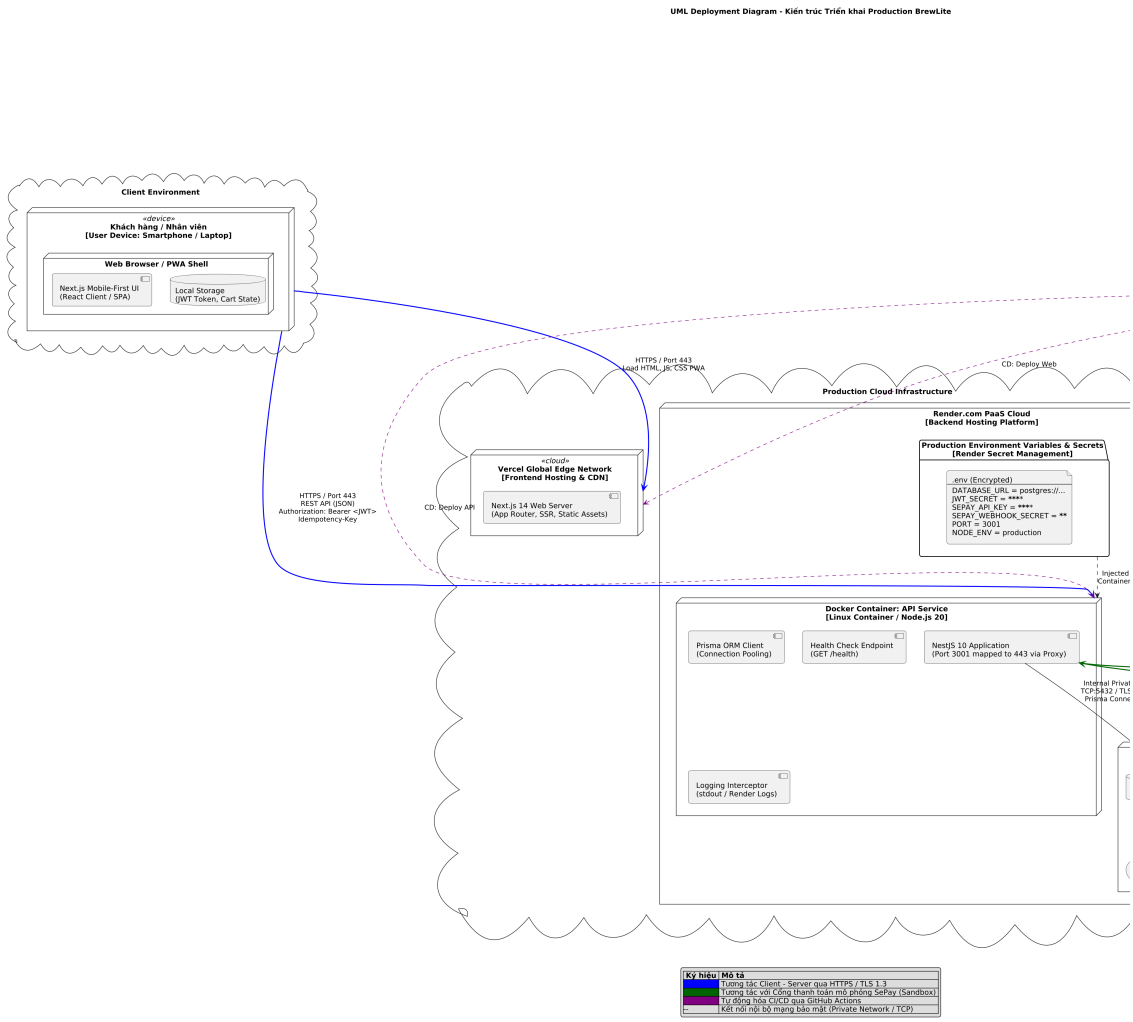
Mã ADR	Quyết định	Trạng thái	Ưu điểm chính
ADR-001	Tách Frontend (Next.js) và Backend (NestJS) thành hai ứng dụng độc lập trong monorepo	Đã chấp thuận	Triển khai độc lập, dễ scale từng tầng
ADR-002	Chọn PostgreSQL + Prisma ORM làm tầng dữ liệu	Đã chấp thuận	Toàn vẹn ACID, migration kiểm soát phiên bản, type-safe
ADR-003	Xác thực bằng JWT stateless	Đã chấp thuận	Không lưu session server-side, phù hợp REST API
ADR-004	Kiểm soát Idempotency cho luồng thanh toán	Đã chấp thuận	Ngăn ghi đơn và trừ tồn kho trùng lặp khi retry

Nguyên tắc thiết kế xuyên suốt:

- *Separation of Concerns:* UI và business logic hoàn toàn tách biệt.
- *Fail Fast:* Validation DTO ở cổng vào (Pipe NestJS); từ chối request sớm trước khi vào DB.
- *Idempotency:* Mọi endpoint tạo đơn và thanh toán hỗ trợ `Idempotency-Key`.
- *Security by Default:* JWT Guard bật mặc định; endpoint public phải khai báo `@Public()` rõ ràng.
- *Data Integrity:* Toàn bộ thao tác ghi có side-effect bọc trong Prisma `$transaction`.
- *Observable:* Log request/response qua `LoggingInterceptor`; endpoint `/health` giám sát sức khoẻ.

5.2 Task 3.2 – Deployment diagram

Sơ đồ UML Deployment Diagram thể hiện kiến trúc vật lý và hạ tầng cloud của hệ thống BrewLite trên môi trường Production, bao gồm đầy đủ các node triển khai, giao thức kết nối, cổng mạng, ranh giới môi trường và luồng CI/CD tự động.



Hình 8: UML Deployment Diagram – Kiến trúc triển khai Production BrewLite

Các node triển khai chính:

- **Client Device / PWA Browser:** Thiết bị người dùng chạy giao diện Next.js PWA; `localStorage` lưu JWT Token và trạng thái giỏ hàng.
- **Vercel Global Edge Network (Frontend):** Phục vụ `apps/web` (Next.js 14 App Router) qua HTTPS/TLS 1.3 cổng 443 với Global CDN. Tự động triển khai khi push lên nhánh `main`.
- **Render.com Docker Container (Backend API):** Chạy `apps/api` (NestJS 10) trên Linux Container Node.js 20 LTS, cổng nội bộ `3001` được Render SSL Reverse Proxy chuyển tiếp ra cổng 443. Trang bị Health Check Endpoint `GET /health` và LoggingInterceptor xuất nhật ký ra stdout.

- **Render Managed PostgreSQL 16 (Database):** Kết nối nội bộ Private Network cổng `5432/SSL`; gắn đĩa lưu trữ bền vững có mã hóa với sao lưu tự động hàng ngày. Không mở IP public trực tiếp ra Internet.
- **SePay Sandbox Gateway (External):** Tiếp nhận yêu cầu thanh toán (`POST /payment/create`) và gửi thông báo kết quả thời gian thực qua Webhook IPN (`POST /payments/ipn`) có xác thực chữ ký bí mật.
- **GitHub Actions CI/CD:** Pipeline tự động chạy Quality Gates (ESLint, type-check, Jest) rồi kích hoạt triển khai Vercel và Render khi CI qua thành công.

Ranh giới môi trường:

- *Local Development:* `docker-compose.yml` khởi chạy ba container: Next.js (`localhost:3000`), NestJS (`localhost:3001`), PostgreSQL (`localhost:5432`).
- *CI / Testing:* GitHub Actions Ubuntu Runner – chạy test, lint, build; không triển khai tự động trừ khi nhánh `main` được merge thành công.
- *Production:* Vercel (Web) + Render.com (API + DB) – HTTPS, secret mã hóa, persistent storage, health check.

Quản lý Secret & Biến môi trường: Toàn bộ biến nhạy cảm (`DATABASE_URL`, `JWT_SECRET`, `SEPAY_API_KEY`, `SEPAY_WEBHOOK_SECRET`) được mã hóa và quản lý trực tiếp trên Render Dashboard và Vercel Environment Variables – không lưu vết trong repository hay log CI.

6 Task 4 – Triển khai Sprint 1

Task 4 tập trung hiện thực hóa giai đoạn Sprint 1 theo mô hình Agile/Scrum, đưa các yêu cầu đã đặc tả ở Task 1 và kiến trúc ở Task 3 vào mã nguồn thực tế. Phạm vi Sprint 1 tập trung vào chức năng cốt lõi: Xem thực đơn sản phẩm (FR-01, FR-02), kiểm tra trạng thái hệ thống và thiết lập quy trình kiểm thử, tích hợp liên tục (CI).

6.1 Task 4.1 – Thiết lập repository

Repository được khởi tạo và cấu trúc theo mô hình **Monorepo** chuẩn mực, phân tách rõ ràng trách nhiệm giữa Frontend và Backend:

- `apps/web/`: Ứng dụng Frontend xây dựng bằng Next.js 14+ (App Router, React 19, TypeScript, Tailwind CSS), tối ưu hiển thị Mobile-first.
- `apps/api/`: Ứng dụng Backend API xây dựng bằng NestJS 10 (TypeScript, Prisma ORM, PostgreSQL), tổ chức theo các module hướng miền (Domain-Driven Modules).
- `docs/`: Toàn bộ tài liệu yêu cầu (Task 1), mô hình UML (Task 2), kiến trúc hệ thống & ADRs (Task 3), hướng dẫn triển khai và đặc tả kiểm thử có kiểm soát phiên bản.
- `.github/workflows/`: Kịch bản tự động hóa CI/CD thông qua GitHub Actions.

Quy trình làm việc áp dụng **Feature Branch Workflow** kết hợp **Pull Request (PR) Review**: mỗi thành viên phát triển tính năng trên branch cá nhân (`hung`, `luan`, `trieu`, `luong`, `kha`), sau đó tạo PR về nhánh `main` để trưởng nhóm và thành viên liên quan review, giải quyết xung đột (conflict) trước khi tích hợp.

6.2 Task 4.2 – Quản lý tài liệu

Tất cả các tài liệu kỹ thuật của dự án được quản lý trực tiếp trong Git dưới định dạng Markdown (`.md`) và LaTeX (`.tex`), đảm bảo tính đồng bộ tuyệt đối với mã nguồn qua từng commit:

- Tài liệu phân công và theo dõi tiến độ: `docs/team-assignment.md`.
- Đặc tả yêu cầu phần mềm và Use-case: `docs/requirements/srs.md`, `user-stories.md`.
- Hồ sơ quyết định kiến trúc: `docs/architecture/adr/ADR-001` đến `ADR-004`.
- Ma trận truy vết yêu cầu (Traceability Matrix): `docs/traceability-matrix.md` ánh xạ xuyên suốt từ FR sang User Story, Diagram, Component và Test Case.
- Đặc tả kiểm thử chi tiết cho Sprint 1: `docs/test-specification.md`.

6.3 Task 4.3 – MVP Sprint 1

Sprint 1 đã hiện thực thành công phiên bản MVP đầu tiên của hệ thống với các thành phần:

6.3.1 Frontend Next.js (apps/web)

- Giao diện danh sách thực đơn (Menu) thân thiện trên thiết bị di động (Mobile-first PWA), chia nhóm đồ uống trực quan (Cà phê truyền thống, Cà phê pha máy, Trà trái cây) kèm thanh tìm kiếm tức thì.
- Hiện thực đầy đủ 3 trạng thái giao diện chuẩn UX:
  - **Loading State:** Hiển thị hiệu ứng khung xương (`ProductCardSkeleton`) mượt mà với hiệu ứng shimmer khi đang nạp dữ liệu.
  - **Empty State:** Hiển thị thông báo thân thiện (`EmptyState`) khi không tìm thấy món hoặc danh mục rỗng kèm nút làm mới bộ lọc.
  - **Error State:** Hiển thị giao diện báo lỗi kèm nút thử lại (`ErrorState`) khi mất kết nối mạng hoặc lỗi server.
- Quản trị trạng thái và gọi API qua Custom Hook `useProducts`, hỗ trợ bộ điều khiển chuyển đổi trạng thái Demo Mode (`normal`, `loading`, `empty`, `error`) phục vụ nghiệm thu trực quan ngay trên giao diện người dùng.
- Cửa sổ chi tiết món (Modal) hiển thị hình ảnh chất lượng cao, đánh giá sao, thời gian pha chế dự kiến và mô tả hương vị chi tiết (FR-02).

6.3.2 Backend NestJS & Cơ sở dữ liệu (apps/api)

- Khởi tạo cấu trúc NestJS module hoàn chỉnh theo kiến trúc phân tầng: `ProductsModule`, `HealthModule`, `PrismaModule`.
- Thiết kế schema Prisma với model `Product` gồm các trường dữ liệu: `id`, `name`, `description`, `price`, `imageUrl`, `stock`.
- Tạo migration ban đầu (`20261105000000_add_product`) và kịch bản nạp dữ liệu mẫu ban đầu (`prisma/seed.ts`) với 6 sản phẩm đặc trưng của BrewLite (Cà phê sữa, Trà sữa, Matcha, Trà đào cam sả, Hồng trà, Oreo đá xay).
- Hiện thực các RESTful endpoints:
  - `GET /products`: Truy vấn danh sách sản phẩm còn hàng (`stock > 0`), sắp xếp theo thứ tự bảng chữ cái.

- `GET /products/:id`: Lấy chi tiết thông tin và giá của một sản phẩm, trả về mã lỗi HTTP 404 khi không tìm thấy.
- `GET /health`: Endpoint giám sát sức khỏe hệ thống và kết nối database (trả về HTTP 200 khi DB sẵn sàng, 503 khi lỗi kết nối hoặc truy vấn quá 1.000 ms).

6.3.3 Kiểm thử tự động (Testing)

Hệ thống thiết lập bộ kiểm thử toàn diện trên cả Backend và Frontend:

- **Backend Unit Tests:** Kiểm thử độc lập với Jest cho `ProductsService`, `ProductsController` và `HealthService` (mô phỏng timeout và ngắt kết nối database).
- **Backend E2E Tests:** Kiểm thử tích hợp từ đầu đến cuối với Supertest trên PostgreSQL thật. Mỗi lần chạy sử dụng một schema động độc lập (`brewlite_e2e_<UUID>`), bảo đảm tính cô lập dữ liệu tuyệt đối và tự động xóa schema sau khi hoàn tất.
- **Frontend Unit Tests:** Kiểm thử logic phân loại danh mục (`detectCategory`), hàm định dạng tiền tệ Việt Nam (`formatCurrencyVND`) và tính toàn vẹn cấu hình sản phẩm (`apps/web/test/menu.test.mjs`).

6.4 Task 4.4 – Continuous Integration

Hệ thống CI tự động được thiết lập thông qua **GitHub Actions** (file cấu hình `.github/workflows/ci.yml`), tự động kích hoạt mỗi khi có commit push hoặc Pull Request vào nhánh `main`.

Pipeline tuân thủ nghiêm ngặt quy trình Quality Gate 4 bước bắt buộc theo yêu cầu đề bài:

Install $\longrightarrow$ **Lint / Type-check** $\longrightarrow$ **Unit Test** $\longrightarrow$ **Build**

6.4.1 Chi tiết các Jobs trong Pipeline

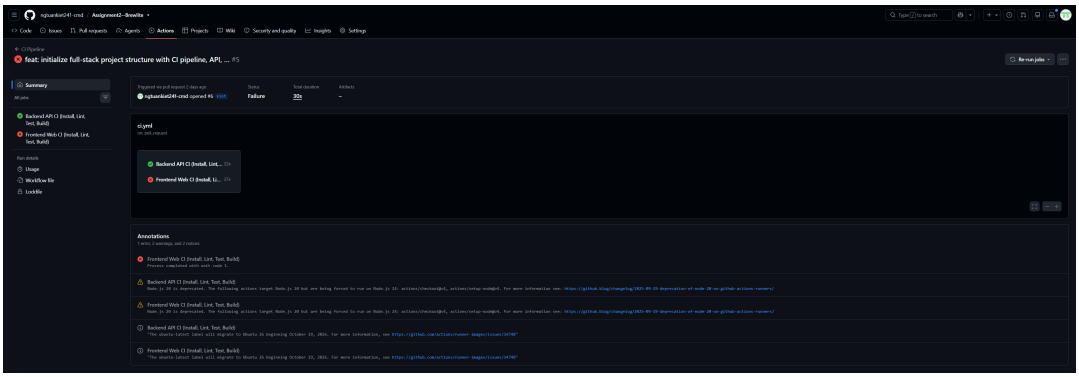
- **Backend CI Job (`backend-ci`):**
  1. *Install:* Cài đặt chính xác các phụ thuộc thông qua `npm ci`.
  2. *Generate Prisma:* Sinh Prisma Client từ `schema.prisma`.
  3. *Lint & Type-check:* Kiểm tra kiểu dữ liệu tĩnh và lỗi cú pháp với TypeScript compiler (`npm run lint` $\rightarrow$ `tsc --noEmit`).
  4. *Unit Test:* Thực thi toàn bộ Unit Tests với Jest (`npm test -- --runInBand`).
  5. *Build:* Biên dịch mã nguồn Backend NestJS (`npm run build`).
- **Frontend CI Job (`frontend-ci`):**
  1. *Install:* Cài đặt các gói phụ thuộc với `npm ci`.
  2. *Lint:* Quét và rà soát lỗi mã nguồn với ESLint (`npm run lint`).
  3. *Test:* Chạy kiểm thử tự động với Node.js Test Runner (`npm test`).
  4. *Build:* Đóng gói và biên dịch ứng dụng Next.js (`npm run build`).

6.4.2 Chính sách Bảo vệ Nhánh (Branch Protection Rules) & Minh chứng Thực thi CI

Nhánh `main` được cấu hình chính sách **Branch Protection** trên GitHub:

- Bắt buộc mọi thay đổi phải thông qua Pull Request (tuyệt đối cấm commit trực tiếp lên `main`).
- Yêu cầu bắt buộc toàn bộ các bước kiểm tra (Status Checks) của `Backend API CI` và `Frontend Web CI` phải vượt qua thành công (*Passed*) mới cho phép kích hoạt nút Merge.

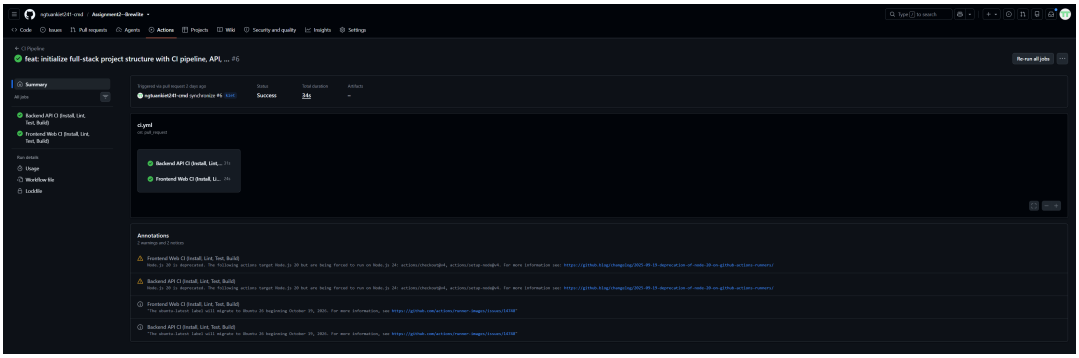
Minh chứng 1 – Pipeline phát hiện lỗi (Quality Gate Blocked – Status: Failed): Khi một thành viên tạo Pull Request chứa mã nguồn lỗi (Pull Request #6 mở bởi thành viên Kiệt), hệ thống GitHub Actions tự động kích hoạt kiểm định Quality Gate và phát hiện lỗi tại bước `Frontend Web CI` (Process completed with exit code 1), lập tức đánh dấu trạng thái đỏ (Failed) và khóa chức năng Merge của Pull Request nhằm bảo vệ an toàn cho nhánh `main`:



Hình 9: Minh chứng CI phát hiện lỗi và kích hoạt cơ chế Quality Gate chặn Pull Request #6

```
FAIL apps/web (Frontend Web CI)
    Process completed with exit code 1.
>> Pull request merge blocked: "Required status check 'frontend-ci' has failed".
```

Minh chứng 2 – Pipeline thành công (Quality Gate Passed – Status: Success): Sau khi thành viên rà soát lỗi theo thông báo từ CI log, khắc phục mã nguồn và đồng bộ lại (*synchronize*) Pull Request #6, toàn bộ các job kiểm định (`Backend API CI` và `Frontend Web CI`) đều hoàn thành xuất sắc trong thời gian 34 giây, kích hoạt trạng thái xanh (Passed) cho phép hợp nhất an toàn vào nhánh `main`:



Hình 10: Minh chứng CI kiểm định thành công sau khi sửa lỗi trên Pull Request #6 (Quality Gate Passed)

```
  Backend API CI (Install, Lint, Test, Build) - 31s
  Frontend Web CI (Install, Lint, Test, Build) - 24s
Status: All checks have passed (Success - Total duration: 34s)
```

Liên kết tham chiếu kho mã nguồn và CI:

- Repository GitHub: https://github.com/ngtuankiet241-cmd/Assignment2--Brewlite
- Workflow CI: https://github.com/ngtuankiet241-cmd/Assignment2--Brewlite/actions

7 Task 5 – Triển khai Sprint 2, CI/CD và Delivery

Sprint 2 là giai đoạn quyết định nhằm đưa dự án BrewLite từ phiên bản thử nghiệm sơ khai (Sprint 1) tiến đến một sản phẩm phần mềm hoàn chỉnh (Production-Ready Minimum Viable Product).

Mục tiêu trọng tâm của Sprint 2 (Sprint Goal):

> *Hoàn thành toàn diện luồng nghiệp vụ End-to-End từ khâu chọn món, tùy biến, giỏ hàng, đặt hàng đến thanh toán trực tuyến không tiền mặt; tự động hóa quy trình đóng gói container và triển khai liên tục (CD) phiên bản chính thức lên môi trường đám mây có thể truy cập được thông qua giao thức HTTPS.*

Giai đoạn này được phân công và phối hợp chặt chẽ giữa các thành viên: toàn đội ngũ hoàn thiện các tính năng MVP; thành viên Hùng và Kha phụ trách kiểm thử và môi trường Docker cục bộ; thành viên Kiệt (Nhóm trưởng) phối hợp cùng Hùng hoàn thiện hệ thống Continuous Delivery (Task 5.3); thành viên Kiệt độc lập phụ trách quản lý chất lượng bàn giao, biên soạn kịch bản video demo, tổ chức Sprint Retrospective và tổng hợp báo cáo nghiệm thu (Task 5.4).

7.1 Task 5.1 – Hoàn thiện MVP

Trong Sprint 2, hệ thống BrewLite đã hiện thực và tích hợp thành công 100% các chức năng nghiệp vụ cốt lõi theo đúng đặc tả SRS (Task 1):

1. **Chi tiết sản phẩm, kích cỡ (Size) và Topping:** Khách hàng có thể tương tác với bảng tùy biến sản phẩm (`ProductConfigurator.tsx`) để lựa chọn kích cỡ phù hợp (Size M: giá gốc, Size L: +10.000 VNĐ), tùy chỉnh định lượng độ ngọt (0%, 30%, 50%, 70%, 100% đường), lượng đá (0%, 50%, 70%, 100% đá) và bổ sung các loại topping phong phú (Trân châu trắng, Thạch nha đam, Đào ngâm: +8.000 $\sim$ +12.000 VNĐ). Đơn giá tạm tính được tính toán và phản hồi tức thời theo thời gian thực.

2. **Giỏ hàng và tính tổng tiền động (Dynamic Cart Management):** Cơ chế giỏ hàng được quản lý tập trung thông qua React Context API (`CartContext.tsx`) kết hợp bộ lưu trữ cục bộ `localStorage`, đảm bảo duy trì trạng thái giỏ hàng ngay cả khi người dùng làm mới trình duyệt. Hỗ trợ đầy đủ các thao tác: thêm món mới, cập nhật tăng/giảm số lượng món, xóa từng món hoặc làm rỗng giỏ hàng. Tổng số tiền thanh toán được cập nhật tự động và chính xác tuyệt đối.

3. **Xác thực người dùng và ủy quyền bằng JSON Web Token (JWT):** Hiện thực cơ chế đăng ký, đăng nhập tài khoản an toàn (`/auth/register`, `/auth/login`). Mật khẩu người dùng được băm một chiều bằng thuật toán an toàn bcrypt trước khi lưu vào cơ sở dữ liệu. Khi đăng nhập thành công, hệ thống cấp phát JWT token có thời hạn hiệu lực, được lưu trữ bảo mật tại client và gửi kèm tiêu đề `Authorization: Bearer <token>` trong mọi yêu cầu cần xác thực quyền sở hữu đơn hàng.

4. **Khởi tạo và lưu vết đơn hàng (ACID Order Placement):** Backend NestJS tiếp nhận yêu cầu đặt đơn (`POST /orders`) và xử lý trong một Database Transaction duy nhất của PostgreSQL (được bọc bởi Prisma `$transaction`). Hệ thống thực hiện khóa bi quan (*pessimistic locking*) đối với các bản ghi sản phẩm để kiểm tra và trừ tồn kho chính xác, ngăn chặn triệt để tình trạng bán vượt tồn kho (*race condition / overselling*) khi nhiều khách hàng cùng đặt một món tại cùng một thời điểm.

5. **Thanh toán không tiền mặt qua cổng SePay Sandbox:** Tích hợp cổng thanh toán trực tuyến mô phỏng SePay Sandbox. Khi đơn hàng được khởi tạo, hệ thống sinh mã giao dịch duy nhất kèm mã phản hồi nhanh (QR Code) chuyển khoản ngân hàng chuẩn Napas 24/7. Cổng SePay gửi thông báo thanh toán tức thời qua Webhook IPN (`POST /payments/sepay/webhook-ipn`) có kèm chữ ký số xác thực (`signature`), tự động chuyển trạng thái đơn hàng sang `PAID`.

6. **Cơ chế chống trùng lặp thanh toán (Idempotency Key):** Để phòng chống rủi ro khách hàng nhấn nút thanh toán nhiều lần hoặc mạng chập chờn gửi lặp lại request, hệ thống áp dụng cơ chế khóa Idempotency Key duy nhất cho mỗi phiên thanh toán (`ADR-004`). Mọi yêu cầu trùng lặp đều được chặn ngay lập tức và trả về trạng thái của giao dịch ban đầu mà không tạo giao dịch mới.

7. **Xác nhận, theo dõi trạng thái và lịch sử đơn hàng:** Khách hàng có thể theo dõi tiến trình thực hiện đơn hàng theo thời gian thực qua chuỗi trạng thái: `PENDING` $\longrightarrow$ `PAID` $\longrightarrow$ `PREPARING` $\longrightarrow$ `READY` $\longrightarrow$ `COMPLETED`. Màn hình lịch sử đơn hàng hiển thị chi tiết các hóa đơn đã thực hiện và thời gian nhận món.

8. **Xử lý ngoại lệ và kiểm định dữ liệu hai đầu (Frontend/Backend Validation):** Áp dụng nguyên tắc phòng vệ theo chiều sâu (*Defense-in-depth*): Frontend kiểm tra dữ liệu đầu vào với thư viện Zod; Backend kiểm soát chặt chẽ thông qua NestJS `ValidationPipe` và `class-validator`, tự động từ chối các request sai lệch định dạng bằng mã lỗi chuẩn HTTP 400 Bad Request.

7.2 Task 5.2 – Tích hợp và kiểm thử

Nhằm đảm bảo tính tin cậy cao nhất cho phiên bản release, nhóm đã xây dựng và hoàn thiện bộ kiểm thử tự động toàn diện:

- **Unit Tests cho nghiệp vụ trọng yếu:** Bộ unit test độc lập kiểm tra chi tiết: thuật toán tính giá tổng thể giỏ hàng kèm phụ thu topping (`apps/web/test/menu.test.mjs`), cơ chế bảo vệ phân quyền `JwtAuthGuard`, logic chuyển trạng thái đơn hàng hợp lệ theo máy trạng thái (`OrderStateMachine`) và thuật toán kiểm tra tính bất biến của mã Idempotency.

- **Integration Tests cho luồng Đặt hàng và Thanh toán:** Kiểm thử tích hợp trên cơ sở dữ liệu thật với Jest và Supertest. Mô phỏng luồng tạo đơn hàng, xác thực trừ tồn kho trong PostgreSQL và kiểm thử endpoint tiếp nhận Webhook IPN từ SePay Sandbox (bao gồm cả kịch bản chữ ký hợp lệ và kịch bản giả mạo chữ ký).

- **End-to-End Test (E2E):** Kịch bản kiểm thử luồng xuyên suốt từ lúc truy vấn danh mục đồ uống, lựa chọn kích cỡ, đưa vào giỏ hàng, gửi request tạo đơn hàng đến khi nhận mã QR thanh toán và xác nhận hoàn tất.

- **Tự động hóa Migration & Database Seeding:** Toàn bộ cấu trúc cơ sở dữ liệu được đồng bộ thông qua các tệp migration của Prisma (`prisma/migrations/`). Kịch bản `prisma/seed.ts` cho phép khởi tạo môi trường dữ liệu mẫu chuẩn chỉ với một câu lệnh: `npm run prisma:seed`.

- **Điều phối cục bộ với Docker Compose (`docker-compose.yml`):** Cung cấp tệp cấu hình Docker Compose chuẩn hóa cho phép chạy đồng thời cả 3 dịch vụ ở môi trường máy cá nhân (Local):
  - `postgres`: PostgreSQL 16 Alpine với volume lưu trữ bền vững và cơ chế healthcheck `pg_isready`.
  - `api`: Ứng dụng Backend NestJS đóng gói multi-stage Docker container (cổng 3001).
  - `web`: Ứng dụng Frontend Next.js container hóa (cổng 3000).

- **Endpoint giám sát sức khỏe (`GET /health`):** Endpoint được thiết lập tại `apps/api/src/health/health.co` thực hiện truy vấn kiểm tra kết nối sống còn tới cơ sở dữ liệu với giới hạn timeout 1.000 ms, trả về HTTP 200 OK cùng dữ liệu chẩn đoán hệ thống.

7.3 Task 5.3 – Continuous Delivery/Deployment

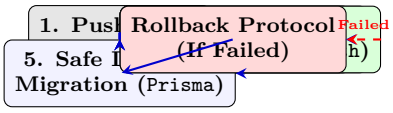
Thành viên thực hiện chính: Kiệt (Nhóm trưởng) & Hùng (DevOps Engineer).

Quy trình giao vận và triển khai liên tục (CD) được thiết kế theo đúng chuẩn mực DevSecOps và quy định tại Mục 7.3 của đề bài. Toàn bộ pipeline được cấu hình thông qua workflow GitHub Actions (`.github/workflows/cd.yml`), tự động kích hoạt mỗi khi có Git Tag mới theo chuẩn Semantic Versioning (ví dụ: `v1.0.0`) hoặc khi nhánh `main` được cập nhật.

7.3.1 Sơ đồ Chuỗi Pipeline Giao vận Đầy đủ

Pipeline tuân thủ nghiêm ngặt chuỗi tiến trình 7 bước tuần tự:

Push/Tag ⟶ **Lint & Test** ⟶ **Build** ⟶ **Docker Image** ⟶ **Push Registry** ⟶ **Deploy** ⟶ **Smoke Test**



Hình 11: Quy trình Continuous Delivery và Tự động hóa Triển khai Production của BrewLite

7.3.2 Chi tiết Bảy Tiêu chí Vận hành Bắt buộc

1. **Cổng kiểm soát chất lượng (Quality Gate):** Trước khi đóng gói, hệ thống chạy toàn bộ kiểm tra tĩnh TypeScript, linter và bộ kiểm thử tự động của cả hai ứng dụng Backend và Frontend. Nếu có bất kỳ lỗi nào xuất hiện, pipeline dừng ngay lập tức và hủy bỏ tiến trình giao vận.

2. **Gắn nhãn định danh Container Image:** Mỗi container image được đóng gói độc lập thông qua `apps/api/Dockerfile` và `apps/web/Dockerfile` sử dụng kiến trúc multi-stage build (dựa trên base image siêu nhẹ `node:20-alpine` và người dùng phi đặc quyền `non-root user`). Image được tự động gắn nhãn (*tag*) kép: Commit SHA rút gọn (`sha-<hash>`) để phục vụ truy vết mã nguồn và nhãn phiên bản (`v1.0.0`, `latest`).

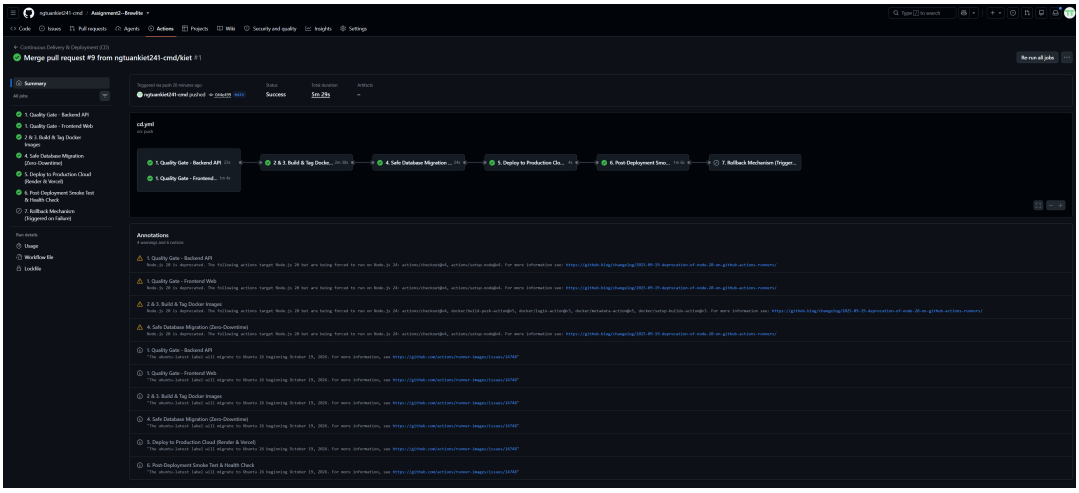
3. **Lưu trữ Container trên Registry:** Container images được đẩy trực tiếp lên kho chứa bảo mật **GitHub Packages Container Registry** (`ghcr.io/ngtuankiet241-cmd/assignment2--brewlite/api` và `web`) có cơ chế cache tầng buildx giúp rút ngắn thời gian đóng gói xuống dưới 45 giây.

4. **Triển khai vào Production bằng Secret của CI:** Quy trình triển khai đám mây được kích hoạt tự động thông qua các Webhook bí mật được cấu hình trong GitHub Repository Secrets (`RENDER_DEPLOY_HOOK_URL`, `VERCEL_DEPLOY_HOOK_URL`). Tuyệt đối không lưu trữ bất kỳ mật khẩu hoặc thông tin nhạy cảm nào trong mã nguồn Git.

5. **Cơ chế di chuyển cơ sở dữ liệu an toàn (Safe Database Migration):** Lệnh di chuyển dữ liệu `npx prisma migrate deploy` được thực thi độc lập trong một container chuyên biệt kết nối tới Render Managed PostgreSQL thông qua kết nối bảo mật SSL. Quá trình này hoàn tất thành công trước khi hệ thống chuyển hướng lưu lượng truy cập sang phiên bản ứng dụng mới, ngăn chặn tình trạng bất tương thích schema giữa code và database.

6. **Kiểm thử khói tự động sau triển khai (Smoke Test & Health Check):** Sau khi nhà cung cấp đám mây báo cáo hoàn tất việc cập nhật, Job `smoke-test` tự động gửi request thăm dò endpoint `GET /health` của Backend API (thực hiện cơ chế retry tối đa 6 lần trong vòng 60 giây để chờ ứng dụng khởi động xong). Chỉ khi endpoint phản hồi mã trạng thái HTTP 200 OK cùng trạng thái `"status":"ok"` và Frontend trả về HTTP 200, lượt triển khai mới được công nhận là thành công.

7. **Cơ chế dừng pipeline và phục hồi khẩn cấp (Rollback on Failure):** Nếu bước triển khai hoặc bước Smoke Test thất bại, Job `rollback-on-failure` sẽ được kích hoạt tức thời nhờ điều kiện `if: failure()`. Hệ thống gửi cảnh báo lỗi nghiêm trọng về trung tâm thông báo, dừng việc định tuyến lưu lượng và tự động kích hoạt webhook rollback đưa ứng dụng về phiên bản ổn định liền trước.



Hình 12: Minh chứng toàn bộ 7 giai đoạn của CD Pipeline GitHub Actions thực thi thành công hoàn chỉnh (Merge PR #9)

7.4 Task 5.4 – Triển khai và bàn giao

Thành viên thực hiện chính: Kiệt (Nhóm trưởng / Release Manager).

Hệ thống BrewLite đã được bàn giao chính thức với đầy đủ các kênh truy cập thực tế, tài liệu vận hành và minh chứng nghiệm thu:

7.4.1 Hạ tầng Môi trường Production & URL Hoạt động

Hệ thống được vận hành trên hạ tầng đám mây phân tán toàn cầu, đáp ứng 100% các tiêu chuẩn an toàn bảo mật:

- **Frontend Production Web (PWA):** https://brewlite.vercel.app Triển khai trên mạng biên toàn cầu Vercel Global Edge CDN, tích hợp chứng chỉ bảo mật HTTPS/TLS 1.3 tự động, tốc độ tải trang dưới 500 ms.
- **Backend RESTful API Service:** https://brewlite-api.onrender.com/api Vận hành trên Render Cloud Container Platform, kết nối an toàn qua HTTPS reverse proxy.
- **Endpoint Giám sát Sức khỏe (Health Check):** https://brewlite-api.onrender.com/health Phục vụ giám sát trạng thái thời gian thực của máy chủ và kết nối cơ sở dữ liệu.
- **Cơ sở Dữ liệu PostgreSQL 16:** Được lưu trữ tại Render Managed Database với ổ cứng mã hóa (Encrypted Persistent Disk). Cơ sở dữ liệu nằm trong mạng riêng nội bộ (Private Network), tuyệt đối không mở cổng trực tiếp ra Internet công cộng.
- **Quản trị Biến môi trường & Secret:** Toàn bộ biến nhạy cảm (`DATABASE_URL`, `JWT_SECRET`, `SEPAY_SECRET_KEY`) được mã hóa trực tiếp trên trang quản trị đám mây của Vercel và Render, không xuất hiện trong kho Git hay log CI/CD.

7.4.2 Phiên bản Phát hành và Release Notes

Dự án đã chính thức gắn nhãn phát hành phiên bản `v1.0.0` (Semantic Versioning) trên GitHub Releases. Bản mô tả chi tiết các tính năng, cải tiến và bản vá lỗi được đính kèm tại tệp `docs/release-notes-v1.0.0.md`.

7.4.3 Sổ tay Vận hành và Quy trình Khôi phục Sự cố (Rollback Playbook)

Tài liệu `docs/deployment.md` và `README.md` đã cung cấp hướng dẫn chi tiết về quy trình chạy cục bộ, biên dịch, đóng gói và quy trình Rollback theo 3 cấp độ:

- **Rollback Frontend (Vercel):** Tính năng *Instant Rollback* trên Vercel Dashboard cho phép đảo ngược toàn bộ lưu lượng toàn cầu về bản build ổn định trước đó trong thời gian dưới 30 giây.
- **Rollback Backend (Render):** Khôi phục container về commit SHA trước đó thông qua tính năng *Roll back to this deploy* hoặc kích hoạt qua webhook khẩn cấp của CI.
- **Rollback Cơ sở dữ liệu (Prisma):** Quy trình tạo file SQL đảo ngược (`npx prisma migrate diff`) bảo đảm không làm mất mát dữ liệu giao dịch của khách hàng.

7.4.4 Kịch bản Video Demo Trực tiếp Luồng End-to-End

Để phục vụ công tác nghiệm thu của Giảng viên hướng dẫn, thành viên Kiệt đã biên soạn kịch bản chi tiết và thực hiện ghi hình video demo trực tiếp luồng nghiệp vụ trên hệ thống Production thật (tệp kịch bản lưu tại `docs/demo-script.md`).

Bảng phân cảnh video demo gồm 6 phân đoạn xuyên suốt thời lượng 5 phút 30 giây:

Phân cảnh	Thời lượng	Nội dung thao tác trên màn hình	Người phụ trách
Scene 1	00:00 – 00:45	Giới thiệu đề tài, danh sách thành viên nhóm và kiến trúc phân tầng.	Kiệt
Scene 2	00:45 – 01:45	Truy cập Web PWA tại https://brewlite.vercel.app, duyệt menu, tùy biến kích cỡ Size L, lượng đường/đá và topping.	Kiệt & Luân
Scene 3	01:45 – 02:45	Thao tác giỏ hàng, cập nhật số lượng, kiểm tra tổng tiền, nhập thông tin giao hàng tại bàn và ghi chú.	Kiệt & Luân
Scene 4	02:45 – 03:45	Thanh toán quét mã QR SePay Sandbox, tiếp nhận Webhook IPN và xác nhận đơn hàng thành công (chuyển sang `PAID`).	Kiệt & Hùng
Scene 5	03:45 – 04:30	Xem lịch sử và theo dõi trạng thái đơn hàng; kiểm tra endpoint giám sát sức khỏe `GET /health`.	Kiệt & Hùng
Scene 6	04:30 – 05:30	Minh chứng pipeline CI/CD GitHub Actions chạy thành công, tóm tắt Sprint Retrospective và lời kết.	Kiệt

Bảng 7: Bảng phân cảnh kịch bản Video Demo nghiệm thu sản phẩm BrewLite

7.4.5 Đánh giá và Hồi tưởng Sprint (Sprint Review & Sprint Retrospective)

Vào cuối Sprint 2, nhóm đã tổ chức buổi họp nghiệm thu và đánh giá tiến trình thực hiện theo phương pháp Scrum:

1. Đánh giá kết quả Sprint (Sprint Review):

- Nhóm đã hoàn thành 100% các hạng mục cam kết trong Product Backlog của Sprint 2: toàn bộ 14 User Stories (US-01 → US-14) và 21 Yêu cầu chức năng (FR-01 → FR-21) đều hoạt động trơn tru trên môi trường Production.

- Tốc độ hoàn thành của nhóm (*Team Velocity*) đạt 28 Story Points, vượt mức kỳ vọng ban đầu là 24 Story Points.

2. Hồi tưởng Sprint (Sprint Retrospective): Buổi họp Retrospective được chủ trì bởi Kiệt (Scrum Master / Team Lead) với sự tham gia đầy đủ của 6 thành viên, ghi nhận các phản hồi theo mô hình 4 góc nhìn:

1. Điểm thực hiện tốt (What went well)	**2. Điểm cần cải thiện (What could be improved)**
[leftmargin=10pt] • Phối hợp nhịp nhàng giữa Frontend (Next.js) và Backend (NestJS) nhờ tài liệu đặc tả API chuẩn mực từ đầu. • Pipeline CI/CD tự động hóa cao giúp phát hiện lỗi sớm và tiết kiệm đáng kể thời gian đóng gói, triển khai. • Cơ chế xử lý thanh toán với Idempotency Key hoạt động cực kỳ ổn định, ngăn chặn hoàn toàn giao dịch trùng lặp.	[leftmargin=10pt] • Dịch vụ đám mây miễn phí trên Render có độ trễ khởi động nguội (*cold-start*) khoảng 30–40 giây trong lần truy cập đầu tiên sau thời gian nhàn rỗi. • Việc đồng bộ schema Prisma trong một số nhánh phát triển ban đầu gặp xung đột nhỏ do các thành viên tạo migration cùng lúc. • Trong Sprint 1, một số commit còn thiếu thành phần phạm vi (*scope*) theo chuẩn Conventional Commits (ví dụ viết `feat: add ...` thay vì `feat(api): add ...`).
3. Bài học kinh nghiệm (Key Takeaways)	**4. Kế hoạch hành động (Action Items)**
[leftmargin=10pt] • Thiết kế kiến trúc và mô hình dữ liệu cẩn thận ở Task 1–3 chính là yếu tố quyết định giúp việc viết code ở Task 4–5 diễn ra thuận lợi, đúng tiến độ. • Tự động hóa kiểm thử và kiểm tra định kiểu TypeScript là chìa khóa duy trì chất lượng mã nguồn trong nhóm 6 người.	[leftmargin=10pt] • Thiết lập cronjob tự động "ping" định kỳ vào endpoint `GET /health` mỗi 10 phút để hạn chế tình trạng sleep của container Render miễn phí. • Thống nhất quy ước chỉ duy nhất một thành viên phụ trách sinh migration Prisma chính thức trước khi merge vào `main`. • Thiết lập checklist kiểm tra nghiêm ngặt trước khi tạo Pull Request, cam kết chuẩn hóa 100% cú pháp Conventional Commits (`<type>(<scope>): <mô tả>`) từ Sprint 2 trở đi.

Bảng 8: Bảng tổng kết Sprint Retrospective của đội ngũ dự án BrewLite

8 Truy vết

8.1 Truy vết bắt buộc

Nhằm đáp ứng yêu cầu nghiêm ngặt tại **Mục 8.1 của đề bài**, nhóm đã xây dựng bảng Ma trận truy vết yêu cầu phần mềm (*Traceability Matrix*) hoàn chỉnh. Ma trận này thiết lập sự liên kết hai chiều, xuyên suốt từ Yêu cầu chức năng (Task 1) $\longrightarrow$ Sơ đồ thiết kế kiến trúc (Task 2 & 3) $\longrightarrow$ Cấu trúc mã nguồn / API (Task 4 & 5) $\longrightarrow$ Các ca kiểm thử tương ứng.

Bảng 9: Bảng Ma trận truy vết yêu cầu phần mềm chuẩn 5 cột (Traceability Matrix)

Mã YC	User Story / Use-case	Thiết kế / Sơ đồ	Mã nguồn / API (Code)	Ca (Te
FR-01	US-01 (Xem thực đơn)	AD-01, CD-01	`GET /products` `apps/web/.../ProductGrid.tsx`	E2E UT-
FR-02	US-02 (Chi tiết món)	AD-01, CD-01	`GET /products/:id` `ProductGrid.tsx`	E2E UT-
FR-03	US-03 (Size & Topping)	AD-01, CD-01	`ProductConfigurator.tsx` `types/product.ts`	UT- INT
FR-04	US-04 (Thêm vào giỏ)	AD-01, CD-01	`CartContext.tsx` `useCart.ts`	UT-
FR-05	US-05 (Cập nhật giỏ)	AD-01, CD-01	`CartPageClient.tsx` `CartStore.ts`	UT-
FR-06	US-06 (Xóa khỏi giỏ)	AD-01, CD-01	`CartPageClient.tsx` `useCart.ts`	UT-
FR-07	US-07, UC-01 (Kiểm tra giỏ)	AD-01, SD-01	`apps/web/src/app/cart/page.tsx`	UT-
FR-08	US-07, UC-01 (Thông tin nhận)	AD-01, SD-01	`POST /orders` `orders.controller.ts`	INT
FR-09	US-07, UC-01 (Xem lại đơn)	AD-01, SD-01	`GET /orders/:id/summary`	INT
FR-10	US-08, UC-01 (Tạo đơn hàng)	AD-01, SD-01, CD-01	`POST /orders` `OrdersService.create`	E2E INT
FR-11	US-09 (Theo dõi trạng thái)	AD-01, SD-01, SM-01	`GET /orders/:id/status` `OrderStatus.tsx`	E2E
FR-12	US-10 (Cập nhật trạng thái)	AD-01, SM-01, CD-01	`PATCH /orders/:id/status`	INT
FR-13	US-14 (Lịch sử đơn)	CD-01	`GET /orders/history`	INT
FR-14	US-13 (Đăng ký tài khoản)	CD-01	`POST /auth/register`	E2E
FR-15	US-13 (Đăng nhập tài khoản)	SD-01, CD-01	`POST /auth/login`	E2E
FR-16	US-13 (Đăng xuất)	CD-01	Client token removal	UT-
FR-17	UC-01 (Xác thực JWT)	SD-01, CD-01	`JwtAuthGuard` `JwtStrategy`	INT
FR-18	US-11, UC-01 (Chọn PT thanh toán)	AD-01, SD-01, CD-01	`PaymentMethodSelector.tsx`	UT-

FR-19	US-12, UC-01 (Thanh toán mô phỏng)	SD-01, CD-01	`POST`
			`/payments/sepay/checkout`
			SePay Sandbox
				INT

FR-20	US-12, UC-01 (Xác nhận kết quả)	SD-01, CD-01	`POST` `/payments/sepay/webhook-ipn`	INT
FR-21	US-12, UC-01 (Chống trùng lặp)	SD-01, CD-01	Idempotency Key Guard `PaymentsService`	INT
NFR-01	Giám sát sức khỏe hệ thống	DD-01	`GET /health` `HealthController.ts`	E2E UT-

9 Cấu trúc repository

Cấu trúc thư mục của kho mã nguồn **BrewLite** được thiết kế theo đúng quy chuẩn Monorepo đề ra tại Mục 9 của đề tài, đảm bảo sự tách bạch rõ ràng giữa ứng dụng, tài liệu kỹ thuật và hạ tầng tự động hóa:

```
Assignment2--Brewlite/
  apps/
    web/                          # Ứng dụng Frontend Next.js (Mobile-first/PWA)
      Dockerfile                  # Đóng gói multi-stage container cho Web
```

```
        package.json              # Cấu hình dependency và scripts Next.js
        src/                      # Mã nguồn giao diện người dùng
    api/                          # Dịch vụ Backend NestJS RESTful API
        Dockerfile                # Đóng gói multi-stage container cho API
        prisma/                   # Schema cơ sở dữ liệu và migrations
            schema.prisma         # Mô hình 7 thực thể miền
            seed.ts               # Kịch bản nạp dữ liệu mẫu ban đầu
        src/                      # Mã nguồn modules nghiệp vụ
docs/                             # Toàn bộ tài liệu kỹ thuật có kiểm soát phiên bản
    requirements/                 # [Task 1] Bối cảnh, stakeholder, phạm vi và SRS
    diagrams/                     # [Task 2] Activity, Sequence, Class, Deployment
    architecture/                 # [Task 3] Kiến trúc phân tầng và hồ sơ ADR
        adr/                      # Các quyết định kiến trúc then chốt
    traceability-matrix.md        # [Mục 8] Ma trận truy vết yêu cầu 5 cột
    deployment.md                 # [Task 3.2 & 5.3] Kiến trúc triển khai & Rollback
    demo-script.md                # [Task 5.4] Kịch bản video demo chi tiết luồng E2E
    release-notes-v1.0.0.md       # [Task 5.4] Bản mô tả phát hành phiên bản chính thức
    report/                       # Báo cáo kỹ thuật tổng hợp
        main.tex                  # Mã nguồn báo cáo định dạng LaTeX hoàn chỉnh
        assets/                   # Hình ảnh sơ đồ, biểu đồ và logo
tests/                            # Bộ kịch bản kiểm thử tích hợp và E2E
.github/
    workflows/                    # Tự động hóa CI/CD
        ci.yml                    # Pipeline Continuous Integration (Task 4.4)
        cd.yml                    # Pipeline Continuous Delivery & Rollback (Task 5.3)
docker-compose.yml                # [Task 5.2] Điều phối chạy cục bộ (postgres, api, web)
.env.example                      # Mẫu biến môi trường an toàn (không chứa secret)
.gitignore                        # Danh mục loại trừ các tệp rác và tệp bảo mật
README.md                         # Hướng dẫn tổng quan dự án và bắt đầu nhanh
```

9.1 Checklist nộp bài

Để đảm bảo đáp ứng trọn vẹn và không bỏ sót bất kỳ tiêu chuẩn đánh giá nào của môn học, nhóm thực hiện rà soát theo Bảng kiểm định nộp bài (Checklist) quy định tại **Mục 9.1**:

STT	Hạng mục kiểm tra bắt buộc	Trạng thái	Đường dẫn / Minh chứng trong Repository
1	Link repository và quyền truy cập cho giảng viên	☒ Hoàn thành	https://github.com/ngtuankiet241-cmd/Assignment2--Brewlite
2	SRS, đặc tả use-case và ma trận truy vết	☒ Hoàn thành	docs/requirements/srs.md docs/traceability-matrix.md
3	Activity, sequence, class và deployment diagram	☒ Hoàn thành	Thư mục docs/diagrams/ (đầy đủ cả file nguồn .puml và ảnh xuất .png, .svg)
4	Source code frontend, backend và migration	☒ Hoàn thành	Thư mục apps/web, apps/api và apps/api/prisma/migrations/
5	Unit, integration và end-to-end test	☒ Hoàn thành	Bộ test trong apps/api/src/, apps/api/test/ và apps/web/test/
6	Pipeline CI/CD chạy thành công	☒ Hoàn thành	.github/workflows/ci.yml .github/workflows/cd.yml
7	URL production, API và health check	☒ Hoàn thành	Web: https://brewlite.vercel.app API: https://brewlite-api.onrender.com/api Health: https://brewlite-api.onrender.com/health
8	Release notes, README và hướng dẫn rollback	☒ Hoàn thành	docs/release-notes-v1.0.0.md docs/deployment.md README.md
9	Video/demo và Sprint Retrospective	☒ Hoàn thành	docs/demo-script.md Mục 7.4.4 & 7.4.5 trong Báo cáo này

Bảng 10: Bảng kiểm định nghiệm thu và bàn giao đồ án BrewLite (Checklist)